



東南大學

本科毕业设计（论文）报告

面向 MoE 模型的片上内存总线统一

虚拟化与预取优化联合设计

学 号:	06022122
姓 名:	王博涵
学 院:	电子科学与工程学院
专 业:	电子科学与技术
指导教师:	单伟伟 江哲
起止日期:	2026. 1–2026. 6

2026 年 6 月 1 日

东南大学毕业（设计）论文独创性声明

本人声明所呈交的毕业（设计）论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得东南大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

论文作者签名：_____  _____ 日期：2026 年 5 月 30 日

东南大学毕业（设计）论文使用授权声明

东南大学有权保留本人所送交毕业（设计）论文的复印件和电子文档，可以采用影印、缩印或其他复制手段保存论文。本人电子文档的内容和纸质论文的内容相一致。除在保密期内的保密论文外，允许论文被查阅和借阅，可以公布（包括刊登）论文的全部或部分内容。论文的公布（包括刊登）授权东南大学教务处办理。

论文作者签名：_____  _____ 导师签名：_____

日期：2026 年 5 月 30 日 日期：2026 年 6 月 17 日

东南大学本科毕业论文（设计）AI 工具使用情况说明表

课题名称	面向 MoE 模型的片上内存总线统一虚拟化与预取优化联合设计		
学 号	06022122	姓 名	王博涵
是否使用生成式人工智能	☒ 是 ☐ 否		
工具、版本号	使用范围	使用过程	章节（页码）
GPT-5	☐ 文本生成及内容修改 ☒ 数据、图表分析、代码调试 ☐ 其他：请具体说明	使用 AI 进行代码报错的修改辅助和若干重复化代码(寄存器初始化等)的自动生成；将实验数据和图表需求喂给 AI 自动生成绘图代码,但架构图和建模图均为 visio 手动绘制;使用 AI 进行论文的格式、错别字、语病检查并进行手动修改还有英文摘要的辅助生成。	第二章第四章实验的柱状图折线图;第三章的代码;全文的格式、错别字还有语病
学生诚信申明	本人郑重声明，上述关于生成式人工智能使用情况的陈述真实无误，已对使用此类技术的所有细节进行了全面且诚实的报告。本人深知学术诚信的重要性，如有任何隐瞒或虚假之处，愿承担学术不端行为带来的相关惩处。 学生签名：王博涵 2026 年 5 月 30 日		
指导教师意见	意见： 经审阅，该生已如实说明生成式 AI 工具使用情况，使用范围主要为文字润色、表达优化、资料整理和代码调试辅助，未替代学生完成论文核心研究、实验分析及结论形成，符合学术规范和学校相关要求。同意该说明。 指导教师签名： 2026 年 6 月 17 日		

摘 要

随着 AI 与大模型发展，混合专家模型（Mixture of Experts, MoE）被大量应用到大规模语言模型上。MoE 利用少量专家激活来提高模型规模，但由于 Token 在各个专家之间分布不平衡，导致不同的专家有不同的访存需求，易发生 SRAM Bank 冲突、带宽低以及访存延迟等问题，从而产生访存受限（Memory-Bound）。

本文提出一种针对 MoE 推理的片上存储系统优化框架 MemDomain，在此基础上利用统一访存总线、存储空间虚拟化以及预取等方法来解决由于动态访存导致的 Bank 冲突问题。文章首先使用周期级模拟器 SCALE-Sim 建立 Bank 冲突感知片上存储模型并选用具有多种大小专家混合模型作为代表性 MoE 工作负载，输入序列长度为 256，隐藏层维度为 384，MoE 层有 8 个不同大小专家。实验表明 MoE 层平均访存延迟是非 MoE 层的 2.12 倍，其访存停顿开销占整个网络 82.86%。

随后，本文提出了一种统一分配 Accumulator 以及 SRAM 访问路径的 MemDomain 结构，加入虚拟 Bank 到物理 Bank 的映射关系并定义 mset、mvin、mvout 等自定义指令，从而实现了软硬件联合下的动态资源调度，在 Buckyball 上完成了 RTL 的设计以及仿真工作。仿真结果显示，相比于静态分配方式，动态分配可以使 MoE 层的推理时间减少 58.19%，访存时间减少 70.21%，而在加入预取窗口之后，整个程序的运行时间可以下降 71.03%，有效访存时间也可以下降 85.69%。综合结果表明，在 28 nm 工艺、2 ns 时钟周期的情况下，标准单元所占的面积会增加 16.06%，逻辑功耗也会提高 12.86%，验证了该方案在性能收益和硬件开销之间具有较好的折中。

关键词：MoE 模型，Memory-Bound，总线统一，存储虚拟化，动态资源分配

ABSTRACT

With the development of artificial intelligence and large-scale models, Mixture of Experts (MoE) has been widely used in large language models. MoE improves model capacity by dynamically activating only part of the experts. However, the uneven distribution of tokens among experts leads to different memory access demands, which may cause SRAM bank conflicts, low bandwidth utilization, and increased memory stalls, forming a memory-bound problem.

This thesis proposes MemDomain, an on-chip memory optimization architecture for MoE inference. It aims to reduce bank conflicts caused by dynamic memory access through a unified memory bus, storage resource virtualization, and cooperative prefetching. First, a bank-conflict-aware on-chip memory model is built based on the cycle-level simulator SCALE-Sim. A representative MoE workload is constructed with reference to the Mixture of Diverse Size Experts (MoDSE). The workload has an input sequence length of 256, a hidden dimension of 384, and eight experts with different sizes. Experimental results show that the average memory access latency of MoE layers is about 2.12 times that of non-MoE layers, and the memory stall overhead of MoE layers accounts for 82.86% of the whole network.

Then, MemDomain unifies the access paths of the Accumulator and SRAM, introduces dynamic mapping from virtual banks to physical banks, and defines customized instructions including mset, mvin, and mvout to support hardware-software collaborative resource allocation. The architecture is implemented and verified at the RTL level on the Buckyball platform. Compared with static allocation, dynamic allocation reduces the inference latency of MoE layers by 58.19% and memory access latency by 70.21%. With prefetching enabled, end-to-end execution cycles are further reduced by 71.03%, and effective memory latency is reduced by 85.69%. Under a 28 nm process and a 2 ns clock period, standard-cell logic area and logic power increase by 16.06% and 12.86%, respectively, showing a good trade-off between performance improvement and hardware overhead.

KEY WORDS: Mixture of Experts, Memory-Bound, Unified Bus, Memory Virtualization, Dynamic Resource Allocation, Prefetching

目 录

摘 要	I
ABSTRACT	II
目 录	III
第一章 绪论	1
1.1 课题背景和意义	1
1.2 研究现状	2
1.2.1 MoE 架构及其在 Transformer 中的应用	2
1.2.2 DNN 加速器中的 Memory-Bound 问题研究	6
1.2.3 仿真工具与研究平台	8
1.3 本文研究内容	9
第二章 MoE 模型访存瓶颈分析与研究动机	11
2.1 实验平台与分析方法	11
2.1.1 基于 Bank-Conflict 的内存模型	11
2.1.2 硬件配置	12
2.1.3 MoDSE 实验网络模型	13
2.2 MoE 访存瓶颈分析	16
2.2.1 MoE 层访存瓶颈特征分析	16
2.2.2 静态资源分配下的访存瓶颈来源分析	20
2.3 本文架构优化动机	22
第三章 总线统一和资源动态虚拟化的方案与实现	24
3.1 基于 Buckyball 平台的 MemDomain 总体架构设计	24
3.2 面向内存域指令的前端解码与任务调度机制	25
3.3 面向多源访存请求的中端统一接入与通道绑定机制	27
3.4 面向虚拟 Bank 映射的后端动态资源管理机制	28
3.5 面向软硬件协同调度的自定义访存指令设计	29
3.6 预取与动态 Bank 分配的协同优化机制	32
第四章 实验验证	34
4.1 总线统一与存储资源虚拟化的正确性验证	34
4.2 动态资源分配机制的优化验证	36
4.3 预取与动态 Bank 分配协同验证	41

4.4 MoDSE 工作负载级实验验证.....	42
4.5 MemDomain 模块综合与实现开销分析.....	45
第五章 总结与展望.....	49
5.1 工作总结.....	49
5.2 工作展望.....	50
参考文献.....	51
致 谢.....	54

第一章 绪论

1.1 课题背景和意义

当今人工智能飞速发展,以 Transformer 为代表的深度神经网络正在对计算体系结构的发展产生巨大影响^{[1][2]}。尤其是在受到大模型、生成式 AI 的双重推动影响下,模型规模和复杂度持续增加^[3],这给底层硬件系统带来了巨大的挑战。因此,专用加速器及其片上存储架构已经成为当前体系结构研究的重要方向,来支撑 AI 的整体系统性能。

近年来,混合专家模型 (Mixture of Experts, MoE) 被大量应用于大规模语言模型和生成式 AI 系统中。通过引入专家路由机制,MoE 模型可以在保证计算量相对较低的情况下大幅度增加模型容量和表达能力,成为支撑超大规模模型的重要手段。但是,MoE 模型在推理时具有不同于一般的稠密模型的行为,在每个专家上分配 Token 数量的不平衡性和专家与专家之间进行 Token 的 All-to-All 通信 (即每个 GPU 都需要把包含 Token 的元数据发送给其他 GPU,同时也接收来自其他 GPU 的元数据),这对于存储系统的访问延迟以及带宽的要求更高。随着计算单元并行度的不断提高,片上存储系统成为限制加速器发展的主要因素。

传统加速器通常采用静态总线和固定存储资源分配方式。对于 MoE 模型而言,这种固定划分方式难以适应专家激活和 Token 分布的动态变化,容易造成部分 Bank 访问压力过大,而其他 Bank 利用率较低,从而加剧 Bank 冲突和资源浪费,这是明显的访存受限 (Memory-Bound) 问题^[4]。因此,为缓解 MoE 推理中的 Memory-Bound 问题,有必要从片上存储结构出发,设计一种更加高效、灵活的内存组织方式。本文提出的片上内存总线统一与存储资源虚拟化架构,通过统一访存路径、引入虚拟 Bank 到物理 Bank 的动态映射,并结合预取协同机制,使片上存储资源能够根据运行时访存需求进行按需分配,从而降低 Bank 冲突、改善不规则访存模式下的资源利用率,并提升加速器整体性能。

此外,本文进一步考虑预取请求与动态 Bank 分配之间的协同关系,通过在数据正式访问前提前完成虚拟 Bank 配置和数据搬入,使预取数据能够结合运行时资源状态映射到合适的物理 Bank,从而减少计算阶段的访存等待,并降低预取过程可能引入的 Bank 竞争。该软硬件协同机制能够进一步发挥统一虚拟化内存架构的性能潜力。本文采用周期级模拟器 SCALE-Sim^{[5][6]}与 RTL 级验证相结合的研究方法,对所提方案进行性能评估,并为后续大模型或通用 AI 加速器的设计提供参考。总而言之,进行面向 MoE 模型的片上内存总线

统一虚拟化与预取优化联合设计研究工作，不仅有明确的实际应用价值，也在加速器体系结构与存储系统优化领域具有重要的研究意义。

1.2 研究现状

随着 AI 的发展，以 Transformer 为代表的大规模深度学习模型已广泛应用于自然语言处理、计算机视觉等各个领域。近年来，大语言模型所涉及的参数数量越来越大，模型越来越复杂，对于底层硬件的要求也越来越高。而传统依赖增大计算单元规模的性能优化方法已经到达极限，影响系统的瓶颈也从计算能力转向存储与数据供给方面。大量研究表明，在高并行度计算平台下，存储系统的带宽、时延和访存冲突是造成整个系统效率低下的主要原因，即 Memory-Bound 问题。而在针对大模型推理加速器的设计中，如何有效地组织片上存储资源、降低访存和充分利用计算单元已成为体系结构需要解决的关键问题。MoE 作为一种目前被广泛使用的大模型设计方法，它所带来的门控网络和专家计算使得模型的参数量大大增加的同时也使得存储访问变得非常不规则，从而使得 Memory-Bound 问题变得更加严重^[7]。因此，对于 MoE 场景下访存瓶颈从系统及实现层面进行研究与优化具有非常重要的理论意义和应用价值，在本章中将围绕 MoE 模型、Memory-Bound 问题以及硬件平台这三方面对现有工作做一个总结。虽然目前已有许多关于 MoE 扩展性和推理加速的研究成果，但是他们大多侧重在模型路由、专家负载均衡、分布式通信以及片外访存优化这几个方面，而对于单芯片加速器内部 SRAM Bank 冲突、片上存储资源静态分配以及 Accumulator 与 SRAM 访问路径分离等问题的关注较少，这也是研究后续工作的出发点。

1.2.1 MoE 架构及其在 Transformer 中的应用

1.2.1.1 Transformer 结构与计算特性

Transformer 使用自注意力机制和前馈网络（Feed-Forward Network, FFN）进行序列建模，在当今大语言模型中占据主导地位。从系统角度来看，Transformer 的计算具有高度并行性，但是也伴随着大量的数据读取和写入到缓存的操作，这对于底层存储有较高的要求。特别是在推理阶段中，模型要在很短的时间内完成多次计算，而存储速度决定了模型的推理速度。

1.2.1.2 MoE 层的基本原理

MoE 模型通过引入多个专家子网络和路由机制，在每次推理过程中仅激活部分专家参与计算，从而在保持模型容量大幅提升的同时有效控制计算量，具体流程如图 1-1 所示。

相较于传统稠密 FFN 层，MoE 层的核心特点在于计算与访存的稀疏性和动态性。已有研究通常从算法层面对 MoE 的稀疏激活和路由稳定性进行优化。例如，传统方法通过 Top-K 算法只对被激活的专家进行加权计算，未激活专家则不参与计算，从而减少无效计算和部分访存开销^[7]。此外，ST-MoE^[8]针对路由过程中可能出现的数值不稳定问题，引入 Router z-loss 对过大的路由 logits 进行约束，以提升训练稳定性并保持模型性能。然而，这类方法主要关注专家选择、路由稳定性和计算稀疏性，并未直接解决路由后 Token 分布不均所导致的片上存储访问热点、SRAM Bank 冲突和访存等待问题。因此，从硬件角度进一步优化 MoE 层的不规则访存行为仍然十分必要。

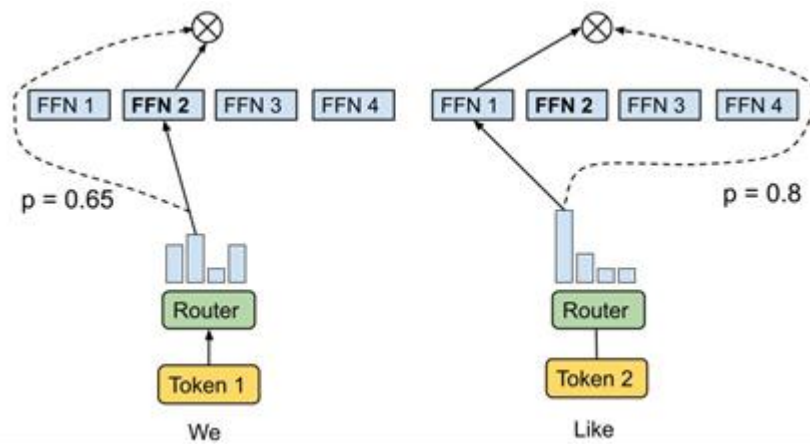


图 1-1 MoE 的算法流程

从硬件角度看，MoE 的稀疏激活机制虽然降低了平均计算量，但也引入了动态且不规则的专家访问模式，使得数据访问难以在运行前被准确预测和规整，进而增加了存储系统调度与资源管理的复杂度。在典型多 GPU MoE 推理系统中，输入词元首先经过门控网络得到专家选择结果，并按照目标专家所在 GPU 进行分组；随后通过 All-to-All 通信完成词元分发，在对应 GPU 上执行专家计算后，再通过 All-to-All 通信将计算结果返回原始设备并完成重组。因此，现有 MoE 系统优化多从系统层面出发，重点关注跨设备通信、专家路由调度、专家放置和负载均衡等问题^[9]。相比之下，单芯片加速器内部由专家计算引发的片上存储访问压力仍缺乏充分研究。因此，有必要从片上存储体系结构角度进一步分析 MoE 推理过程中的访存瓶颈。

1.2.1.3 MoE 在 Transformer 中的应用研究

近年来，MoE 已被广泛应用于 Transformer 架构中，尤其作为 FFN 层的替代方案，用

于支撑超大规模模型的构建。已有研究主要从模型容量扩展、路由策略优化、训练稳定性以及分布式执行效率等角度对 MoE 进行优化。这些工作有效提升了 MoE 模型的规模和计算效率，但其关注重点多集中在算法和系统层面，对单芯片加速器内部的片上访问冲突问题讨论相对较少。

GShard^[10]从大规模模型并行与自动分片的角度出发，将部分 Transformer 块中的 FFN 替换为 MoE 层，并采用 Top-2 门控机制，使不同 MoE 层能够分布在多个设备上执行，从而将 Transformer 模型参数规模扩展到 6000 亿以上，具体架构如图 1-2 所示。该工作证明了 MoE 在扩展模型容量方面的有效性，但其优化重点主要在跨设备并行和路由分配上，而不是片上 SRAM Bank 资源调度。

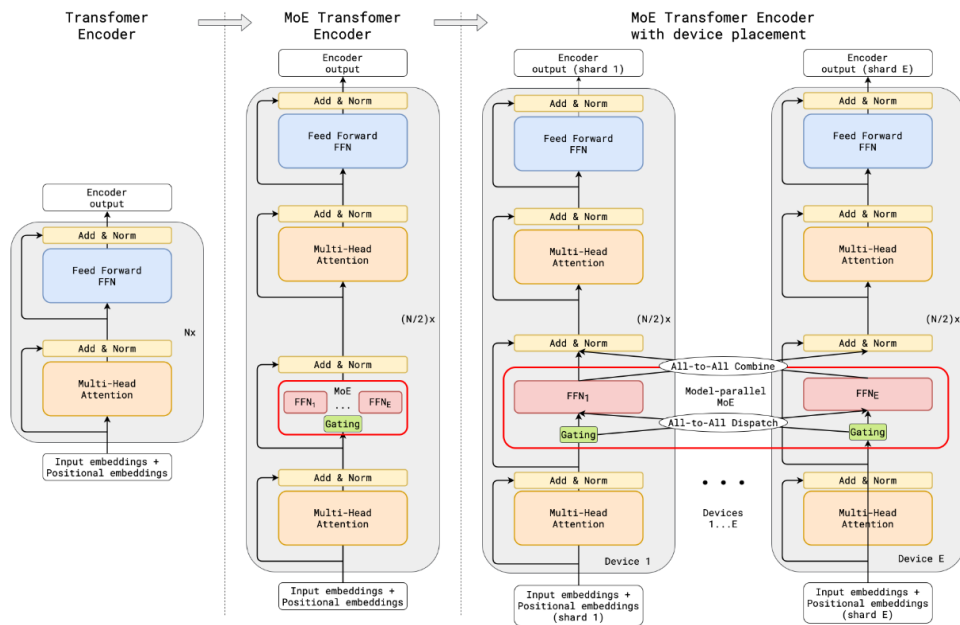


图 1-2 Gshard 的系统架构

Switch-Transformer^[11]则从简化路由机制和提升训练效率的角度出发，将 MoE 路由从 Top-2 简化为 Top-1，使每个 Token 仅被分配到一个专家，从而降低专家计算和通信开销，并结合混合精度训练提升整体训练效率，系统架构如图 1-3 所示。本文所采用的 MoDSE 工作负载在结构上与 Switch-Transformer 中以 MoE 层替代 FFN 层的思想相近，因此可以用于分析 MoE 层在单芯片加速器内部对片上访问系统造成的影响。然而，Switch-Transformer 主要关注路由简化和模型训练效率，并未进一步分析专家负载不均对片上 SRAM Bank 层面引发的访问冲突。

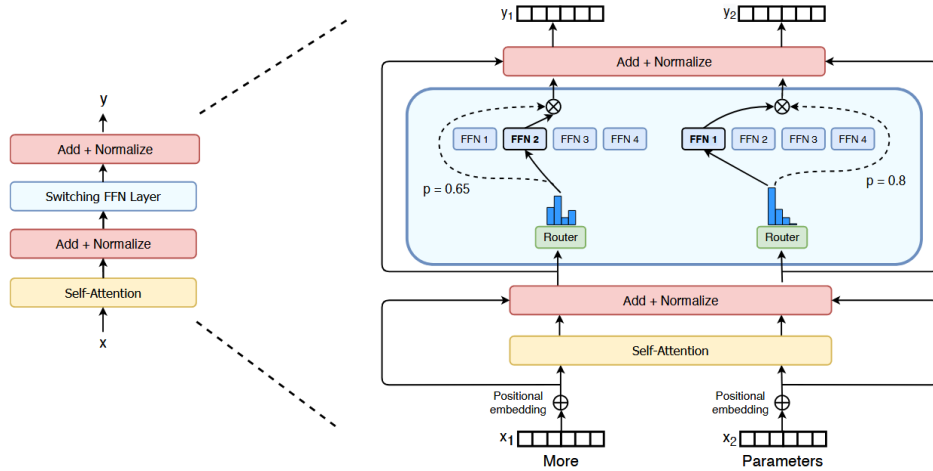


图 1-3 Switch-Transformer 的系统架构

GLaM^[12]进一步从稀疏激活与计算效率提升的角度出发，在 Transformer 层中以 MoE 层替代前馈组件，并通过门控函数为每个输入 Token 从 64 个专家中选择得分最高的两个专家进行加权计算，如图 1-4 所示。该方法在多个自然语言处理任务中取得较好效果，并在推理阶段减少了每个 Token 所需的计算量。尽管如此，GLaM 的重点仍然是通过稀疏激活降低计算开销，而不是针对 MoE 路由后形成的不规则访存请求进行片上存储结构优化。

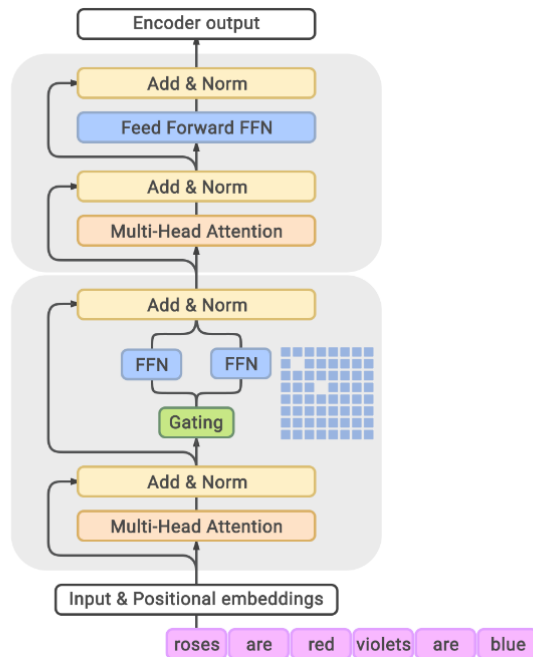


图 1-4 GLaM 的系统架构

综上所述,这些工作主要证明了 MoE 在模型规模扩展和计算效率提升方面的有效性,其优化重点集中在路由机制、专家组织方式、训练稳定性和分布式执行效率上。然而,对于单芯片加速器内部的细粒度片上访问行为,尤其是 SRAM Bank 冲突、不同数据类型之间的资源竞争,以及固定存储划分导致的 Bank 利用率下降问题,现有研究仍缺乏系统分析。因此,本文将研究重点放在 MoE 推理过程中的片上存储系统动态资源管理与微架构优化上。

1.2.2 DNN 加速器中的 Memory-Bound 问题研究

1.2.2.1 Memory-Bound 问题概述与优化研究现状

对于 DNN 加速器而言,Memory-Bound 问题是由于系统速度被数据读取写入延时或者存储带宽所限制而不是被计算能力所限制。在片上存储层次上一般使用多 Bank 组织形式以便能够支持并行操作。但是当多个访存请求试图在同一时间访问同一个 Bank 的情况下就会出现 Bank 冲突,造成访问堵塞并且使得计算单元闲置,大大降低了整个系统的工作效率。所以在资源有限的情况下减少 Bank 冲突是存储体系结构优化中的一个重要研究课题。

在计算单元与存储单元间关系的研究中,Roofline 模型^[13]等经典方法被提出用以描述计算密度、存储带宽以及所能达到的最大性能之间的关系。这对理解在大规模并行计算环境中带宽是影响整体性能的关键因素具有重要意义。而对于这一问题的研究,学者们也分别从硬件实现以及体系结构的角度进行了大量探讨。例如,Asad.A 团队^[14]认为由于计算密度不断增加使得数据传输所需要的时间以及能耗已经远远大于计算本身,在高度并行的神经网络中,存储器带宽和效率成为了进一步提升系统性能的重要障碍。对此,也有许多相关的工作致力于解决这个问题,其中一种是从硬件实现上考虑使用三维存储集成技术(如 Neurocube^[15]、DeepTrain^[16]、QUEST^[17]),利用硅通孔(TSV)将存储器放置到处理器上面从而提高带宽和降低访问延时;而另一种则是研究存内计算(Processing-in-Memory, PIM),即将一部分处理功能移到存储端以减少数据搬移带来的负担,比如 FloatPIM^[18]和 ParaPIM^[19]。

除了对硬件改进外,在架构层面上的研究主要是针对片外存储访问行为进行改善,比如提高数据复用性^[20],或降低预取期间行缓冲区发生冲突的概率从而增强并行访存能力^[21]等,但是这些都是以固定的内存布局以及固定的硬件资源为基础的。总体来看,已有 Memory-Bound 优化方法主要从提高片外带宽、减少数据搬移、提升数据复用率等角度展

开，这些方法能够缓解存储访问带来的系统瓶颈，但大多建立在相对固定的存储层次和资源组织方式之上。对于 MoE 这类运行时访存需求高度变化的负载，仅依靠增加带宽或优化片外访问并不能充分解决片上 Bank 热点、资源闲置以及数据通路竞争等问题。因此，片上存储资源的动态组织与按需分配仍需要进一步研究。

1.2.2.2 MoE 场景下的 Memory-Bound 特性

相比于稠密 DNN，MoE 模型在访存方面表现得更加随机无序。一方面，专家激活是时变的，不同的时刻专家被激活的情况和所接收的 Token 数量可能完全不同，从而造成访存热点的变化；另一方面，由于各个专家间的负载不均也会加剧局部存储资源的访存冲突。因此传统的基于固定划分的片上存储结构难以很好地满足 MoE 的需求，容易出现严重的 Bank 冲突以及带宽不足的问题。

一些现有工作已经注意到了 MoE 给系统执行效率带来的问题，并尝试从负载均衡、访存感知调度和主动内存管理等角度进行优化。Shazeer 等人^[7]从专家负载均衡角度出发，在训练过程中引入辅助损失项，对不同专家之间的负载不均进行软约束，以缓解专家利用率不均和跨设备执行压力。进一步地，METRO 算法^[13]指出 MoE 中 Token 级负载均衡并不等同于访存级负载均衡，因此将单位时间内被唤醒专家及其所需内存大小总和作为优化目标，以降低每秒带宽需求，缓解专家规模差异带来的访存压力。与此类似，微软相关研究^[23]从内存感知路由和权重预取角度出发，认为传统 MoE 推理过程中专家选择信息被利用得较晚，导致专家权重加载延迟较大，因此通过提前暴露专家选择信息来支持主动内存管理和减少带宽竞争。上述工作已经从算法训练、专家调度、路由选择和权重预取等层面缓解了 MoE 的负载不均与访存压力，但其关注重点仍主要位于模型算法或系统调度层面。对于单芯片加速器内部更细粒度的片上存储访问行为，例如 SRAM Bank 冲突、不同数据类型之间的 Bank 资源竞争，以及固定片上存储划分导致的资源利用率下降问题，现有研究仍缺乏针对性的微架构优化。因此，本文进一步从片上存储系统角度出发，研究面向 MoE 推理的动态 Bank 分配与访存路径优化。

总体而言，MoE 场景下的 Memory-Bound 问题不仅表现为片外带宽压力，也体现在单芯片加速器内部的片上存储资源竞争。由于专家激活和 Token 分布具有动态性，固定 Bank 划分方式难以持续匹配不同层、不同专家的访存需求，容易造成部分 Bank 过载而部分 Bank 闲置。与此同时，传统 Accumulator 与 SRAM 分离的数据通路也可能增加访存路径和控制逻辑复杂度。基于上述不足，本文从片上存储微架构出发，研究统一访存通路、虚拟 Bank

映射和动态资源分配机制，以提高 MoE 推理过程中的片上存储利用率并降低 Bank 冲突。

1.2.3 仿真工具与研究平台

针对上述片上存储瓶颈问题，本文需要完成两个层面的研究：一方面，需要通过周期级仿真对不同 Bank 分配策略下的访存冲突和性能变化进行快速评估；另一方面，需要在 RTL 层面验证统一总线、虚拟 Bank 映射和动态资源分配机制是否具有可实现性。

在 NPU 体系结构研究中，不同的仿真工具和验证平台通常承担不同功能。其中，周期级模拟器能够更真实地反映硬件结构对性能的影响，可以进行大规模参数的 Sweep 测试和性能瓶颈的研究；而 RTL 级验证平台更倾向于对硬件模块的实现应用和功能验证，用于测试新的架构是否实现了需要的硬件逻辑需求。因此，本文采用周期级模拟器 SCALE-Sim 进行体系结构级性能分析，并基于 Buckyball 平台完成 MemDomain 结构的 RTL 实现与功能验证。

1.2.3.1 SCALE-Sim v3 仿真平台

SCALE-Sim 是一款面向 DNN 加速器的周期级体系结构模拟器，能够对计算阵列规模、片上/片外存储结构、数据流模式以及访存带宽等关键参数进行建模与分析^{[5][6]}。相比于 RTL 级的硬件仿真，架构模拟器并不关心具体的门级电路实现，而是在不同的架构参数下对系统运行的周期数以及访存行为进行近似估计，因此更有利于定位性能问题及大量参数搜索等。

在本课题中，SCALE-Sim v3 主要用于构建周期级加速器性能模型，根据 GOOGLE TPU v2 风格张量阵列参数化定义片上 Bank 数目、带宽以及资源分配方式，然后通过一系列不同内存配置 Sweep 实验来研究由于访存竞争导致 Stall Cycle 的变化情况，在此基础上对比静态资源分配与动态资源分配这两种方法在不同 Bank 并行度以及访存带宽条件下所带来的影响，进而得出动态资源调度对于解决 Memory-Bound 问题的有效性，以便于以后对片上内存进行改进优化。

1.2.3.2 基于 Gemmini 的 Buckyball 平台

Gemmini^[24]及其扩展平台 Buckyball 提供了一个面向 NPU 的 RTL 级硬件设计与验证平台。与体系结构模拟不同，RTL 级验证直接基于寄存器传输级（Register Transfer Level）硬件描述开展仿真，能够更加真实地反映硬件模块之间的数据传输、流水线控制以及接口时序行为，因此适用于验证硬件设计的功能正确性与工程可实现性。

在本课题中，Buckyball 平台主要用于验证所提出 MemDomain 存储架构的 RTL 级实

现效果。本文基于 Chisel 对统一片上总线、虚拟 Bank 映射以及动态资源分配机制进行硬件实现，并结合 Verilator 仿真与 Design Compiler 综合，对所提出结构的数据通路、控制逻辑以及资源开销进行验证。相比于体系结构模拟阶段的大规模性能分析，RTL 级验证更关注架构方案能否在真实硬件系统中正确运行，从而为所提出方法的工程实现提供支撑。

1.3 本文研究内容

现有 MoE 相关研究已经在模型结构、路由策略、专家负载均衡和分布式通信优化等方面取得了较多成果，Memory-Bound 相关研究也从带宽扩展、数据复用和片外访存优化等角度提出了多种方法。然而，对于 MoE 推理在单芯片加速器内部引发的片上 SRAM Bank 冲突、存储资源静态划分失配以及 Accumulator 与 SRAM 数据路径分离等问题，现有工作仍缺乏针对性的分析与优化。基于此，本文以 MoE 推理过程中的片上访存瓶颈为研究对象，结合周期级仿真与 RTL 级验证，提出一种统一总线与存储资源虚拟化相结合的 MemDomain 架构，以实现片上存储资源的动态分配和 Bank 冲突缓解。全文的工作可以分为三部分。

第一部分是 MoE 模型访存行为建模及性能瓶颈研究，在 MoE 模型推理时存在访存模式非常随机情况下，利用周期级模拟器 SCALE-Sim 对其访存行为进行建模分析。以包括注意力机制、路由机制和多专家前馈网络典型 MoE 为例，从各个级别上对其访存特性进行细致描述。然后着重探讨 MoE 层造成的访存开销对于整个网络影响，对访存瓶颈主要是由于片上 SRAM Bank 发生冲突原因进行评估，其次考察不公平资源分配造成局部 Bank 热点形成及带宽使用降低情况，还有就是多个通路访存结构（Accumulator 与 SRAM）之间隐含资源竞争问题。实验结果表明传统的固定 Bank 划分方法不能满足随机动态专家被选择引起不同访存需求变化，因此需要对片上存储系统进行统一抽象并实现灵活调度来解决此问题。具体实验方案将在第二章中进行详细阐述。

第二部分，在第一部分的基础上提出了一个基于统一总线驱动虚拟化片上内存域（MemDomain）。其主要思想是把 Accumulator 以及 SRAM Bank 到内存访问路径合并到一条总线上来减少多个路径带来的仲裁开销，同时引入虚拟 Bank 的概念对物理 Bank 进行映射及分配并且根据实际的需求来进行调度来提高片上存储利用率，此外定义了用于虚拟化资源管理的操作（例如 mset 操作等）让硬件可以明确知道需要什么样的资源进而实现更加高效合理的存储调度。而在实现上是在 Buckyball 基础上用 Chisel 硬件描述语言完成。具

体架构方案与模块细节将在第三章中进行系统介绍。

第三部分，进行软硬件联合系统级验证及评价工作，在此过程中首先利用基于 Buckyball 平台中的 BBtest 测试工具对新提出的方案进行功能验证，保证其可以良好地支持各种类型的神经网络操作；其次从访存延迟、Stall 周期、Bank 利用率以及系统整体吞吐能力这几个方面入手对动态虚拟化技术对于解决 Memory-Bound 问题所带来的帮助进行考察；最后根据 Design Compiler (DC) 综合的结果对该设计方案所付出的硬件代价进行估算，在性能提升的同时兼顾硬件成本。具体测试将在第四章中进行详细阐述。此外，本文进一步构建预取窗口模型，用于分析预取请求与动态 Bank 分配之间的协同关系。该模型关注数据提前搬运对访存等待的隐藏效果，以及预取数据写入片上 SRAM 时对 Bank 冲突和资源占用的影响。通过比较无预取、静态预取和动态预取等配置，评估统一虚拟化内存架构对预取过程的支持能力。具体测试将在第四章中进行详细阐述。

本文的主要创新点主要有三点：（1）提出了统一总线的片上存储架构，实现多路径访存的整合及简化；（2）采用虚拟 Bank 动态分配的方法来提升存储资源的使用效率以及解决 Bank 冲突问题；（3）设计软硬件协同的存储调度策略，实现灵活高效的片上存储管理。

第二章 MoE 模型访存瓶颈分析与研究动机

2.1 实验平台与分析方法

2.1.1 基于 Bank-Conflict 的内存模型

本论文使用一种针对片上存储访问瓶颈而设计的 Bank-Conflict 内存模型来描述在矩阵加速时由于多个线程同时对同一 Bank 访问造成的影响。不同于以往侧重于缓存命中率以及完整的 DRAM 时序进行建模的方法，在这里我们主要考虑三个影响吞吐的关键因素：Bank 划分、同周期冲突排队以及因为容量不足带来的惩罚。这种方式可以在保证较低计算代价的情况下很好地反映出 Attention 层、Router 层和 MoE 专家层对于内存访问的不同影响，因此也可以作为体系结构设计空间搜索的一个良好估算工具。

从 Bank-Conflict 内存模型抽象角度看，片上访问的对象被划分为输入激活（Input Activation）、权重（Weight）以及输出激活（Output Activation），并且它们都分别对应一个 Bank 集合。设总 Bank 数为 B_{total} ，则三类张量分配满足 $(B_{in} + B_w + B_{out} = B_{total})$ ，且各项均不小于 1。对任意访问地址，采用模运算完成 Bank 映射；当同一周期内多个请求映射到同一 Bank 时，模型依据“每 Bank 每端口每周期最多服务一个请求”的约束进行排队服务。请求的实际服务时刻由请求到达时刻与 Bank 最早可用时刻共同决定，因此冲突会以服务推迟的形式累积，并最终体现为层级 Stall 周期增加。如图 2-1 所示，每一个命令行在地址映射完毕后会以 128 bit/cycle 的带宽进行数据传输，若一个 Bank 有多个命令行，则需要排队等待，这也导致了 Bank 冲突。这说明当可分配的资源足够多时，可以通过动态分配的方式，给虚拟地址分配物理地址来减少命令行排队的负担，减少冲突导致的延时。此外，模型考虑了矩阵乘法后的比特膨胀问题，RTL 实际实现的输出是 32 bit 的数据，对于更多比特的数据，由于带宽的限制，模型将数据分成多次进行传输。这也就导致了更多的延时，因此在实际中尽管输出矩阵的元素规模不是很大，但是需要更多的资源进行存储结果值。

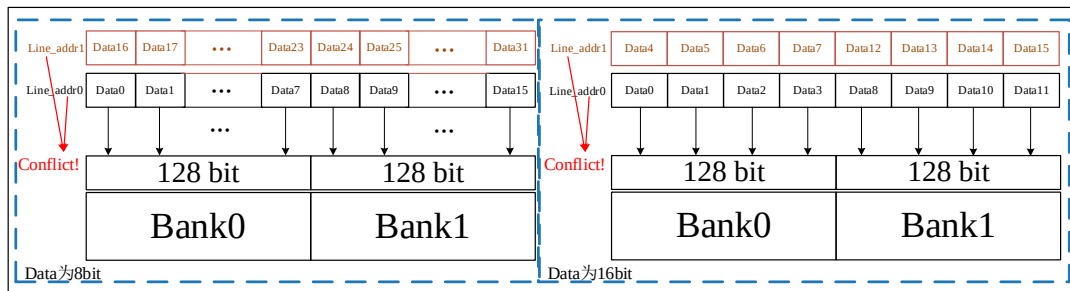


图 2-1 基于 Bank-Conflict 的内存模型原理

为统一刻画带宽与冲突强度，本文将单请求服务时间定义为基础服务周期与容量惩罚周期之和。其中，基础服务周期由数据字长与有效单 Bank 带宽共同决定，并乘以冲突惩罚系数形成最终的无溢出服务开销。若以张量(t)表示三类访问对象之一，其基础项可写为公式 (2.1)：

$$S_t^{base} = p_{conf} \left[\frac{w_t}{BW_{eff}} \right] \quad (2.1)$$

其中 w_t 为张量元素字节数， BW_{eff} 为折算后的每 Bank 带宽， p_{conf} 为冲突惩罚系数。它是决定当容量足够大时每个单独请求所要付出最低代价，在以后惩罚中起到参考标准作用。

当某个张量所分配的 Bank 总容量不足以包含它的工作集大小，在模型中加入额外的 DRAM 罚分来近似表示频繁地从外部读取造成的时间开销，这是容量惩罚周期的定义。即首先求出需要的容量以及可获得的容量之比作为溢出率，然后用对数的形式把溢出程度转化为一个罚分并加上一个下限防止低估严重的溢出情况，那么一个请求总的等待时间为 ($S_t = S_t^{base} + S_t^{DRAM}$)。这样既可以保持由于容量不足造成的性能急剧下降的非线性特点，也可以免于进行整个 DRAM 时序仿真所带来的高成本，在架构级仿真上做到高效又易懂。

对于 Bank 分配策略，本模型支持静态以及动态两种方式，在静态方式下根据配置文件所指定比例分配，在动态方式下从所有符合条件候选集中搜索出能使 Stall 最小化、总体耗时最短并且当两者相同时优先选取与位宽加权容量目标值更为接近 Bank 分配方案作为最终结果。此方法可以根据每层访问特性自动调节 Input Activation、Weight 和 Output Activation 在各 Bank 上占比从而减少热点冲突提高吞吐率。最后输出总周期、冲突 Stall、分张量冲突延迟、容量利用率、溢出字节数以及每个 Bank 利用情况供后续实验研究和参数调整使用。

2.1.2 硬件配置

为了验证本文提出的片上存储架构对 MoE 模型的效果，在此使用 SCALE-Sim 模拟器搭建硬件平台和神经网络的工作负载，在同一平台上比较了不同的存储管理模式对于系统的影响，主要研究片上存储资源的分配对系统的影响，如 Bank 冲突、访存延迟、利用率等方面。

从硬件上，在图 2-2 (a) 所示中，本文使用的是 GOOGLE TPU v2 通用张量核 NPU

参数。其中，二维计算阵列大小是 64×64 ，用于执行矩阵乘运算。片上存储系统总容量为 96 KB，其中 Input Activation、Weight 与 Output Activation 分别分配 32 KB 的存储容量。每一类的数据都采用多个 Bank 的形式进行组织，包括 Input Activation、Weight 以及 Output Activation 都划分为若干个 SRAM Bank，而且每个 Bank 都可以支持单端口或者多端口的操作，Bank 级别的带宽是 128 bit/cycle，Bank 的总数为 24 个，在数据流动上使用了权重固定（Weight Stationary, WS），这样可以最大化地利用权重重用并减少不必要的数据传输开销。

对于预取机制，本文在原有 Bank-Conflict 访存模型基础上扩展轻量级预取窗口模型。该模型通过是否启用预取和预取窗口大小控制未来数据块的提前搬运行为。其中， $W=0$ 表示不启用预取， $W=1$ 表示在当前计算阶段提前为下一阶段所需数据发起预取请求。本文主要预取对象为下一计算阶段所需的 Input Activation 和 Weight，不对 Output Activation 进行预取。为保证实验可比性，不同实验组使用相同硬件参数和相同 MoE 工作负载，仅改变是否启用预取以及是否启用动态 Bank 分配。

对于存储系统的建模，本文使用了 Bank 基础上的冲突建模方法（EnableBankModel）并实现了一种动态资源分配方式（EnableDynamic）。即在静态分配情况下每一种数据类型在开始之前就固定分配一定数量的 Bank，在动态分配的情况下则是依据实际运行过程中的访存情况对 Bank 进行按需调度以及重新分配，以减少因为访存不均所带来的资源闲置。模型中进一步设置 Bank 冲突惩罚参数（BankConflictPenalty），用于近似表示同周期多个请求访问同一 Bank 时由仲裁、排队等待和流水线服务延迟共同造成的额外开销。本文将 BankConflictPenalty 设为 4，表示一次 Bank 冲突会引入 4 个周期的等效惩罚。需要说明的是，该参数属于周期级模型中的抽象建模参数，会影响 Stall Cycles 的绝对数值；但本文所有静态分配、动态分配以及预取实验均采用相同的 BankConflictPenalty 设置，因此实验重点在于比较不同存储管理策略在同一建模条件下的相对性能差异，而不是讨论该参数本身的最优取值。惩罚系数的变化会引起整体 Stall Cycles 的变化，但不同分配策略之间由于 Bank 冲突程度不同而产生的相对差异仍可用于分析片上存储资源分配机制的有效性。总的来说，为突出片上存储系统对整体性能的影响，本文在建模过程中采用惩罚项近似模拟片外存储访问和容量溢出带来的额外延迟，同时对片上 SRAM 的 Bank 映射、冲突排队和访问服务过程进行精细建模，以刻画片上存储行为对系统性能的影响。

为增强实验的可复现性，本文将主要硬件配置、输入规模、路由分布和专家规模统一整理于表 2.1。

2.1.3 MoDSE 实验网络模型

在网络模型方面，如图 2-2（b）所示，本文采用基于 Transformer 的结构，由 Attention 模块、Router 模块以及 MoE-FFN 模块共同组成，并在前馈网络部分引入 Mixture of Diverse Size Experts（MoDSE）模型^[25]作为研究对象。相较于传统 MoE 模型中各专家规模一致的设计，MoDSE 在同一层中引入不同规模的专家，使得各专家的参数数量与计算复杂度存在显著差异。

考虑到模拟器在计算规模与运行时间方面的限制，本文在保持模型结构特征不变的前提下，对原始网络参数进行了等比例缩放，使其能够在周期级模拟环境中高效运行。需要说明的是，本文研究重点并非评估完整大模型的语言生成精度，而是分析 MoE 推理过程中由专家规模差异、Token 分布不均和稀疏激活带来的片上访存不平衡、Bank 冲突以及资源利用率下降问题。因此，在保留 Attention、Router 和多专家 FFN 等关键结构特征的情况下，对网络维度进行缩放仍然能够反映 MoE 工作负载的主要访存特性，并可用于比较不同 Bank 分配策略之间的相对性能差异。缩放后的网络仍保留注意力模块、路由模块以及多专家前馈结构，其中输入序列长度设为 256，隐藏层维度为 384，注意力部分包含标准的 Q、K、V 投影以及相关矩阵运算。

在 MoE 层，本文研究使用 8 个专家，不同的专家其前馈网络大小差别较大。比如一些专家拥有较大的扩展维度（如 1536、1728 等），而部分专家规模较小（如 192、384 等）。这样不同的 Token 在进行路由时就会导致不同的计算及内存访问量，在运行时会出现非常严重的不均衡内存访问。

与传统均匀 MoE 不同，MoDSE 中不同专家的 FFN 扩展维度不同。例如，E0、E2、E4 等专家具有较大的扩展维度，而 E1、E3 等专家规模较小。同时，路由后不同专家接收的 Token 数也不同。本文不单独模拟路由函数的计算过程，而是参考 MoDSE 论文中的专家负载分布，将其按比例缩放并取整后作为本文实验中的代表性路由分布，如图 2-2（b）和表 2.1 所示。需要说明的是，表中各专家 Token 数表示进入专家计算的有效 Token 数，用于构造具有代表性的非均衡 MoE 工作负载；由于比例缩放和取整处理，各专家 Token 数之和与输入序列长度之间可能存在少量差异。本文关注的是专家间 Token 分布不均与专家规模差异对片上访存行为的影响，而不是对某一次具体输入的路由结果进行逐 Token 级复现。

由于 MoE 路由结果本身与输入样本相关，不同输入可能产生不同的专家负载分布，因此各层 Stall Cycles 的绝对数值会随路由分布变化而变化。本文采用固定的代表性路由分布，主要目的是在可复现条件下刻画 MoE 推理中常见的专家负载不均现象，并比较不同存储资源分配策略在相同输入分布下的相对性能差异。只要路由结果存在专家间负载不均，

且不同专家规模存在差异，静态 Bank 分配与实际访存需求之间的失配问题仍然会出现，因此该设置能够支持本文对片上存储资源动态分配机制的分析。

这种设置能够同时体现两类不均衡：一是不同专家接收的 Token 数量不同，导致各专家输入激活访问量不同；二是不同专家的 FFN 扩展维度不同，导致权重规模和输出写回规模不同。因此，相比所有专家规模一致的传统 MoE 结构，MoDSE 更容易暴露片上存储系统中的 Bank 冲突、资源静态划分失配和带宽利用不均问题，更适合作为本文动态 Bank 分配机制的实验工作负载。

```

1 [[general]]
2 run_name = dynamic_24
3
4 [architecture_presets]
5 arrayheight = 64
6 arraywidth = 64
7 ifmapsramszkb = 32
8 filtersramszkb = 32
9 ofmapsramszkb = 32
10 ifmapoffset = 0
11 filteroffset = 10000000
12 ofmapoffset = 20000000
13 bandwidth = 1024
14 dataflow = wd
15 readrequestbuffer = 128
16 writerequestbuffer = 128
17
18 [layout]
19 ifmapcustomlayout = True
20 ifmapsrambankbandwidth = 128
21 ifmapsrambanknum = 8
22 ifmapsrambankport = 1
23 filtercustomlayout = True
24 filtersrambankbandwidth = 128
25 filtersrambanknum = 8
26 filtersrambankport = 1
27 ofmapsrambanknum = 8
28 ofmapsrambankbandwidth = 128
29 ofmapsrambankport = 1
30 [sparsity]
31 sparsitysupport = 1
32 sparserep = ellpack_block
33 optimizedmapping = false
34 blocksize = 8
35 randomnumbergeneratorseed = 40
36
37
38 [run_presets]
39 interfacebandwidth = USER
40 useramulatortrace = False
41 EnableBankModel = True
42 EnableDynamic = True
43 BankConflictPenalty = 4
44 EnableCapacityPenalty = False
45 DRAMPenaltyScale = 2

```

```

1 Layer,M,N,K
2 Attn_Q_proj,256,384,384,
3 Attn_K_proj,256,384,384,
4 Attn_V_proj,256,384,384,
5
6 QKT_head,256,512,64,
7 QKTV_head,256,64,512,
8
9 Attn_O_proj,256,384,384,
10
11 Router_logits,256,8,384,
12
13 MoE-E0-FF1,32,1728,384,
14 MoE-E0-FF2,32,384,1728,
15
16 MoE-E1-FF1,48,192,384,
17 MoE-E1-FF2,48,384,192,
18
19 MoE-E2-FF1,50,1536,384,
20 MoE-E2-FF2,50,384,1536,
21
22 MoE-E3-FF1,24,384,384,
23 MoE-E3-FF2,24,384,384,
24
25 MoE-E4-FF1,34,1152,384,
26 MoE-E4-FF2,34,384,1152,
27
28 MoE-E5-FF1,28,768,384,
29 MoE-E5-FF2,28,384,768,
30
31 MoE-E6-FF1,20,960,384,
32 MoE-E6-FF2,20,384,960,
33
34 MoE-E7-FF1,18,960,384,
35 MoE-E7-FF2,18,384,960,

```

(a) GOOGLE TPU V2 的硬件参数

(b) MoDSE 网络结构

图 2-2 实验硬件配置与网络模型结构

表 2.1 实验工作负载与仿真配置

配置类别	参数	设置
输入规模	输入序列长度	256
输入规模	隐藏层维度	384
注意力模块	Q/K/V 投影	$256 \times 384 \times 384$
注意力模块	QKT_head	$256 \times 512 \times 64$
注意力模块	QKTV_head	$256 \times 64 \times 512$

续表 2.1 实验工作负载与仿真配置

配置类别	参数	设置
路由模块	Router logits	$256 \times 8 \times 384$
MoE 专家数	专家数量	8
路由分布	各专家 Token 数	32、48、50、24、34、28、20、18
专家规模	FFN 扩展维度	1728、192、1536、384、1152、768、960、960
计算阵列	Systolic Array	64×64
片上存储	SRAM 总容量	96 KB
片上存储	Ifmap/Filter/OfmapRAM	32 KB/32 KB/32 KB
Bank 配置	总 Bank 数	24
Bank 配置	单 Bank 带宽	128 bit/cycle
Bank 配置	单 Bank 端口数	1
数据流	Dataflow	Weight Stationary
请求缓冲	Read/Write Request Buffer	128/128
冲突模型	EnableBankModel	True
动态分配	EnableDynamic	True
冲突惩罚	BankConflictPenalty	4

2.2 MoE 访存瓶颈分析

本节基于所提出的 MoDSE 工作负载，在周期级模拟器中对系统性能瓶颈进行分析。通过对不同的网络层在运行过程中所消耗的时间进行统计，如总的执行周期、Stall Cycles 和 Bank 不平衡的情况等，研究各个层次的访存模式的不同之处并找出造成系统整体性能受限因素。在此基础上，首先从网络结构的角度出发分析访存瓶颈在各个部分之间的分布情况，确定系统内主要消耗性能的部分；然后根据存储资源划分方法来探讨导致访存瓶颈出现的原因，以便后续设计合理的改进方案。

2.2.1 MoE 层访存瓶颈特征分析

系统访存瓶颈主要集中于 MoE 层：为分析不同网络层对系统性能的影响，本文对各层的停滞周期（Stall Cycles）、停滞占比（Stall Ratio）以及 Bank 利用不均衡程度（Bank

Imbalance) 进行统计, 并按照模块类型将其划分为 MoE 层与非 MoE 层进行对比分析, 原始统计结果如图 2-3 所示, 详细结果如表 2.2 所示。其中, Stall Cycles 用于刻画由于存储系统无法在期望周期内完成数据供给或写回而导致计算流水线等待的累计开销; Stall Ratio 表示 Stall Cycles 在该层总执行周期中的占比, 用于衡量该层受访存等待影响的程度; Bank Imbalance 则用于描述不同 SRAM Bank 之间访问负载或资源利用是否均衡。

在本文的逐周期建模过程中, 设第 i 条访存请求在无阻塞情况下应于某一时钟周期发起。由于 Bank 冲突、端口数量限制、单次传输带宽限制以及容量溢出惩罚等因素, 该请求的实际服务完成时刻可能被推迟。对于来自不同类型矩阵 (Input Activation、Weight、Output Activation) 的张量 t , 该请求对应的访存延迟可表示为公式 (2.2):

$$D_{t,i} = C_{t,i}^{svc} - C_i^{req} \quad (2.2)$$

其中, $D_{t,i}$ 表示第 i 条请求在张量类型 t 上的访存延迟, $C_{t,i}^{svc}$ 表示服务完成周期, C_i^{req} 表示请求发起周期。由于矩阵计算过程通常需要同时满足输入激活、权重读取以及输出写回等访存约束, 因此当前请求行引入的停滞时间 S_i 由三类访存对象中延迟最大的部分决定, 可表示为公式 (2.3):

$$S_i = \max_t D_{t,i} \quad (2.3)$$

进一步地, 某一网络层的总停滞周期 C_{stall} 可由所有请求行的停滞时间累加得到为公式 (2.4):

$$C_{stall} = \sum_i S_i \quad (2.4)$$

在此基础上, 停滞占比 R_{stall} 定义为公式 (2.5):

$$R_{stall} = \frac{C_{stall}}{C_{total}} \quad (2.5)$$

其中, C_{total} 表示该层完成计算和访存服务所需的总执行周期。 R_{stall} 越接近 1, 说明该层执行时间中大部分周期都消耗在访存等待上, 系统越明显地表现出 Memory-Bound 特征。

对于 Bank Imbalance, 本文采用 Bank 利用率的离散程度进行衡量。设某一网络层中第 b 个 Bank 的访问次数为 A_b , 该层总执行周期为 C_{layer} , 该 Bank 的利用率可表示为公式 (2.6):

$$u_b = \frac{A_b}{C_{layer}} \quad (2.6)$$

在此基础上, 本文使用变异系数 (Coefficient of Variation, CoV) 描述不同 Bank 利用

率之间的不均衡程度，即公式（2.7）：

$$\text{Bank Imbalance} = \frac{\sigma(u)}{\mu(u)} \quad (2.7)$$

其中， $\mu(u)$ 和 $\sigma(u)$ 分别表示所有相关 Bank 利用率集合 u_b 的均值和标准差。Bank Imbalance 数值越大，说明访问请求越集中于少数 Bank，Bank 间负载越不均衡，也更容易形成热点访问和排队等待，从而增加 Stall Cycles；反之，则说明访存请求在不同 Bank 间分布更加均匀，片上存储并行度利用更加充分。

从表 2.2 可以观察到，不同网络层在执行过程中存在显著的性能差异。相比注意力等非 MoE 模块，MoE 层在多个性能指标上均表现出更高的开销，尤其是在 Stall Cycles 与 Bank 利用不均衡程度方面，呈现出更加明显的集中趋势。然而，由于不同层之间的规模与计算特性存在差异，仅通过逐层数据难以直观反映整体规律，因此有必要进一步对相关指标进行统计汇总。

为此，本文对 MoE 层与非 MoE 层的关键性能指标进行了聚合分析，统计结果如表 2.3 所示。结果表明，MoE 层平均每层 Stall Cycles 大约为普通层的 2.12 倍，平均 Bank 利用不均衡程度也大 1.29 倍。而在整个运行期间内，MoE 层占用了大约 82.86%的总 Stall Cycles，因此大部分性能开销都集中在 MoE 相关操作上。

针对非均衡访存的特性分析：造成这种现象的根本原因是 MoE 模型本身的设计，相比非 MoE 层中访存模式较规整、负载分布较均衡的矩阵运算，MoE 层的访存行为具有更强的不规则性。由于路由机制会将不同 Token 动态分配至不同专家，而不同专家在网络规模和访存需求上又存在明显差异，因此当大规模专家接收较多 Token 时，会在短时间内产生大量集中访存请求。这类请求容易映射到部分热点 Bank，使这些 Bank 频繁被访问，而其他 Bank 的利用率较低，从而造成 Bank 间负载不均、访存排队增加和 Stall Cycles 上升。该现象说明 MoE 层的性能瓶颈主要来源于动态专家路由与静态片上存储资源组织之间的不匹配。

进一步地，由各层 Stall Ratio 可以观察到，在本文的周期级 Bank-Conflict 模型和硬件配置下，大部分层的 Stall 占比较高，说明其执行周期中相当一部分由访存等待贡献。需要说明的是，Prefill 阶段由于输入 Token 较多，通常具有较高计算量，并不必然表现为 Memory-Bound；但在本文所构造的 MoDSE 工作负载中，专家路由不均、专家规模差异以及静态 Bank 划分共同导致片上 SRAM Bank 访问出现热点和排队，使得计算单元频繁等待数据供给。因此，本文根据较高的 Stall Ratio，并结合 Bank Imbalance 和 MoE 层 Stall 贡献占比，判断该配置下系统主要受片上存储访问冲突和数据供给延迟限制。

总而言之，当前系统的主要性能瓶颈并非来源于计算资源不足，而是来源于 MoE 场景下动态访存需求与静态存储资源组织之间的不匹配。MoE 层在 Stall Cycles 与 Bank 利用不均衡程度上的显著升高，表明传统静态 Bank 分配方式难以适应动态专家激活所带来的不规则访存模式。因此，后续分析将进一步聚焦 MoE 层，研究访存瓶颈与存储资源分配策略之间的关系，并探索动态资源调度机制对缓解 Bank 冲突问题的作用。

```

LayerID, EnableBankModel, EnableDynamic, EnableCapacityPenalty, bank_conflict_penalty, DRAMPenaltyScale, total_banknum, ifmap_banknum, filter_banknum, ofmap_banknum, allocation_ratio,
bank_capacity_kb, ifmap_total_capacity_kb, filter_total_capacity_kb, ofmap_total_capacity_kb, ifmap_elements, filter_elements, ofmap_elements, ifmap_capacity_utilization,
filter_capacity_utilization, ofmap_capacity_utilization, ifmap_overflow_to_dram, filter_overflow_to_dram, ofmap_overflow_to_dram, ifmap_dram_penalty_cycles_per_request,
filter_dram_penalty_cycles_per_request, ofmap_dram_penalty_cycles_per_request, ifmap_bank_conflict_delay, filter_bank_conflict_delay, ofmap_bank_conflict_delay, total_bank_conflict_delay,
total_cycles, stall_cycles_due_to_bank_conflict,
0, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 98304, 147456, 98304, 3.0, 4.5, 12.0, True, True, True, 0, 0, 0, 195990, 884736, 209808, 1290534, 85175, 69120,
1, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 98304, 147456, 98304, 3.0, 4.5, 12.0, True, True, True, 0, 0, 0, 195990, 884736, 209808, 1290534, 85175, 69120,
2, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 98304, 147456, 98304, 3.0, 4.5, 12.0, True, True, True, 0, 0, 0, 195990, 884736, 209808, 1290534, 85175, 69120,
3, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 16384, 32768, 131072, 0.5, 1.0, 16.0, False, False, True, 0, 0, 0, 13160, 196608, 279744, 489512, 26967, 23400,
4, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 131072, 32768, 16384, 4.0, 1.0, 2.0, True, False, True, 0, 0, 0, 45073, 196608, 34968, 276649, 18525, 14958,
5, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 98304, 147456, 98304, 3.0, 4.5, 12.0, True, True, True, 0, 0, 0, 195990, 884736, 209808, 1290534, 85175, 69120,
6, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 98304, 3072, 2048, 3.0, 0.09375, 0.25, True, False, False, 0, 0, 0, 37224, 0, 228, 37452, 8402, 5727,
7, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 12288, 663552, 55296, 0.375, 20.25, 6.75, False, True, True, 0, 0, 0, 134136, 3981312, 137457, 4252905, 210680, 174717,
8, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 55296, 663552, 12288, 1.6875, 20.25, 1.5, True, True, True, 0, 0, 0, 134136, 3981312, 30540, 4145994, 206711, 170748,
9, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 18432, 73728, 9216, 0.5625, 2.25, 1.125, False, True, True, 0, 0, 0, 21561, 442368, 10056, 473985, 25115, 20832,
10, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 9216, 73728, 18432, 0.28125, 2.25, 2.25, False, True, True, 0, 0, 0, 21306, 442368, 20112, 483786, 26237, 21954,
11, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 19200, 589824, 76800, 0.5859375, 18.0, 9.375, False, True, True, 0, 0, 0, 179160, 3538944, 87744, 3805848, 202271, 167712,
12, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 76800, 589824, 19200, 2.34375, 18.0, 2.34375, True, True, True, 0, 0, 0, 180870, 3538944, 21936, 3741750, 195395, 160836,
13, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 9216, 147456, 9216, 0.28125, 4.5, 1.125, False, True, True, 0, 0, 0, 22896, 884736, 24570, 932202, 45665, 37962,
14, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 9216, 147456, 9216, 0.28125, 4.5, 1.125, False, True, True, 0, 0, 0, 22896, 884736, 24570, 932202, 45665, 37962,
15, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 13056, 442368, 39168, 0.3984375, 13.5, 4.78125, False, True, True, 0, 0, 0, 94554, 2654208, 33210, 2781972, 142829, 118638,
16, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 39168, 442368, 13056, 1.1953125, 13.5, 1.59375, True, True, True, 0, 0, 0, 94590, 2654208, 11070, 2759868, 139553, 115362,
17, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 10752, 294912, 21504, 0.328125, 9.0, 2.625, False, True, True, 0, 0, 0, 52704, 1769472, 55404, 1877580, 92483, 76788,
18, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 21504, 294912, 10752, 0.65625, 9.0, 1.3125, False, True, True, 0, 0, 0, 52704, 1769472, 27702, 1849878, 91349, 75654,
19, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 7680, 368640, 19200, 0.234375, 11.25, 2.34375, False, True, True, 0, 0, 0, 48600, 2211840, 52875, 2313315, 112724, 93825,
20, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 19200, 368640, 7680, 0.5859375, 11.25, 0.9375, False, True, False, 0, 0, 0, 48600, 2211840, 21150, 2281590, 111023, 92124,
21, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 6912, 368640, 17280, 0.2109375, 11.25, 2.109375, False, True, True, 0, 0, 0, 44280, 2211840, 48330, 2304450, 112004, 93285,
22, True, False, False, 4, 2, 24, 8, 8, 8, 8:8:8, 4.0, 32.0, 32.0, 32.0, 17280, 368640, 6912, 0.52734375, 11.25, 0.84375, False, True, False, 0, 0, 0, 44280, 2211840, 19332, 2275452, 110303,
91584,

```

图 2-3 不同网络层性能指标统计原始数据图

表 2.2 不同网络层性能指标统计

LayerName	StallCycles	StallRatio	BankImbalance
Attn_Q_proj	69120	0.811506	9.000000
Attn_K_proj	69120	0.811506	9.000000
Attn_V_proj	69120	0.811506	9.000000
QKT_head	23400	0.867727	15.500000
QKTV_head	14958	0.807449	3.000000
Attn_O_proj	69120	0.811506	9.000000
Router_logits	5727	0.681623	2.906250
MoE-E0-FF1	174717	0.829300	19.875000
MoE-E0-FF2	170748	0.826023	18.750000
MoE-E1-FF1	20832	0.829464	1.687500
MoE-E1-FF2	21954	0.836757	1.968750
MoE-E2-FF1	167712	0.829145	17.414062

续表 2.2 不同网络层性能指标统计

LayerName	StallCycles	StallRatio	BankImbalance
MoE-E2-FF2	160836	0.823133	15.656250
MoE-E3-FF1	37962	0.831315	4.218750
MoE-E3-FF2	37962	0.831315	4.218750
MoE-E4-FF1	118638	0.830630	14.101562
MoE-E4-FF2	115362	0.826654	12.304688
MoE-E5-FF1	76788	0.830293	8.671875
MoE-E5-FF2	75654	0.828186	8.343750
MoE-E6-FF1	93825	0.832343	11.015625
MoE-E6-FF2	92124	0.829774	10.664062
MoE-E7-FF1	93285	0.832872	11.039062
MoE-E7-FF2	91584	0.830295	10.722656

表 2.3 MoE 与非 MoE 层性能指标统计对比

指标	MoE	Non-MoE	倍数
平均 Stall Cycles	96874	45795	2.12x
平均 Bank Imbalance	10.60	8.20	1.29x
Stall 贡献占比	82.86%	17.14%	—

2.2.2 静态资源分配下的访存瓶颈来源分析

在上一小节中已经验证，系统的主要访存瓶颈集中于 MoE 层。进一步地，本节从存储资源组织方式的角度出发，分析该瓶颈产生的根本原因。当前系统采用静态 Bank 分配机制，即在运行前固定划分 Input Activation、Weight 与 Output Activation 所对应的 Bank 数量。然而，不同网络层之间的访存需求存在显著差异，因此固定资源划分方式可能导致部分数据类型出现资源不足，而其他资源处于闲置状态，从而影响整体存储系统效率。

为分析静态分配策略对系统性能的影响，本文在总 Bank 数量与总带宽保持不变的前提下，对 Input Activation、Weight 与 Output Activation 的 Bank 分配比例进行多组调整，并记

录不同配置下 MoE 层的 Stall Cycles 变化情况，实验结果如图 2-4（a）所示。

系统性能对于静态 Bank 分配方式较为敏感：为分析静态 Bank 分配比例对访存性能的影响，本文在相同输入规模、相同路由后 Token 分布以及相同专家规模设置下，仅改变 Input Activation、Weight 和 Output Activation 三类数据对应的 Bank 数量。实验结果如图 2-4（a）所示，当资源分配比例由偏向 Weight/Output Activation 的配置（例如 2:11:11）逐渐变为相对均衡的配置时，MoE 层 Stall Cycles 整体呈现先减小后增大的趋势，并且在不同 Bank 分配比例之间存在明显差异。其中，在部分相对均衡或与当前访存需求较匹配的配置下（例如 7:7:10、7:10:7 和 8:14:2），Stall Cycles 明显降低；而当资源分配严重偏向某一类数据时（例如 14:8:2、18:3:3），其他数据类型可用 Bank 数量减少，其访存请求容易集中到少数 Bank 上，形成新的热点冲突，从而使 Stall Cycles 再次增加。需要说明的是，MoE 层的绝对 Stall Cycles 会受到输入 Token 分布和路由结果的影响。不同输入可能激活不同专家，并产生不同的访存压力分布。为排除输入变化带来的干扰，本文在本组实验中固定路由后 Token 分布和网络规模，仅改变静态 Bank 分配比例。因此，图 2-4（a）中不同配置之间的性能差异主要反映 Bank 资源划分方式对片上访存冲突的影响。由此可见，在相同工作负载条件下，系统性能对于 Bank 分配方式较为敏感，访存效率不仅取决于总 Bank 数和总带宽，也与不同类型数据对应的 Bank 资源比例密切相关。

资源不足产生的 Bank 冲突问题：进一步分析可知，在某一种类型的数据（以 Input Activation 为例）所占资源较少的情况下，大量的访存请求需争夺有限的 Bank 资源而产生严重的 Bank 冲突；随着该种类型数据资源的比例适当地提高（例如由原来的 2:11:11 变为 7:7:10），局部冲突得以缓解，系统的性能也有所改善。但是当资源进一步向这一类数据倾斜的时候（例如变为 14:8:2、18:3:3），另一种数据类型（例如 Weight 或 Output Activation）所使用的 Bank 数量减少，它们的访存请求又集中在少数几个 Bank 中，形成新的热点冲突，使得总的 Stall Cycles 再次升高。这表明系统的性能并不是由某一种类型的数据决定的，而是由各种不同类型的数据之间所占资源的关系决定。过多地倾向于某种类型的资源配置，都会打破整个系统的访存负载均衡，造成局部热点现象严重。

静态分配策略的结构性缺陷：为更好地分析其产生原因，在图 2-4（b）中给出不同的分配比例下 Bank 组成情况。从图可以看出，在极端的静态分配策略下，一种类型的数据几乎占用了所有 Bank，而另一种类型的数据只能使用很少一部分资源，从而使得访存请求的空间分布非常集中。这是因为由于资源分配不合理造成的热点所引起的 Bank 冲突以及由此带来的降低系统性能的问题。另外，在图 2-4（c）中还展示出在不同的静态分配策略下 MoE 每一层的访存冲突敏感度分布。实验结果显示，不同层数目在不同的资源分配情

况下所面临的冲突严重程度差别较大，并且高冲突区域也会因为不同的资源分配而改变位置。这就意味着不同 MoE 层数目对于访存的需求是有很区别的，而固定不变的静态资源组织方式是无法满足所有层的需求。综上所述，当前存在的访存瓶颈并非仅仅是由 Memory-Bound 造成的，而且也与静态资源分配方式不能很好地适应动态访存需求有关。尽管某些静态分配方案可以在某种情况下取得较好的效果，但是它们都有很强的针对某一层或某种场景的要求，不能适用于各种不同的任务负载上。

所以，可以说目前系统的最大访存问题是由于静态 Bank 分配造成的，为了进一步改善系统的性能，需要增加一种 Dynamic Bank 分配方式，在运行时根据实际访存的需求来灵活地调整可用资源，以减少访存冲突并且充分利用所有资源。

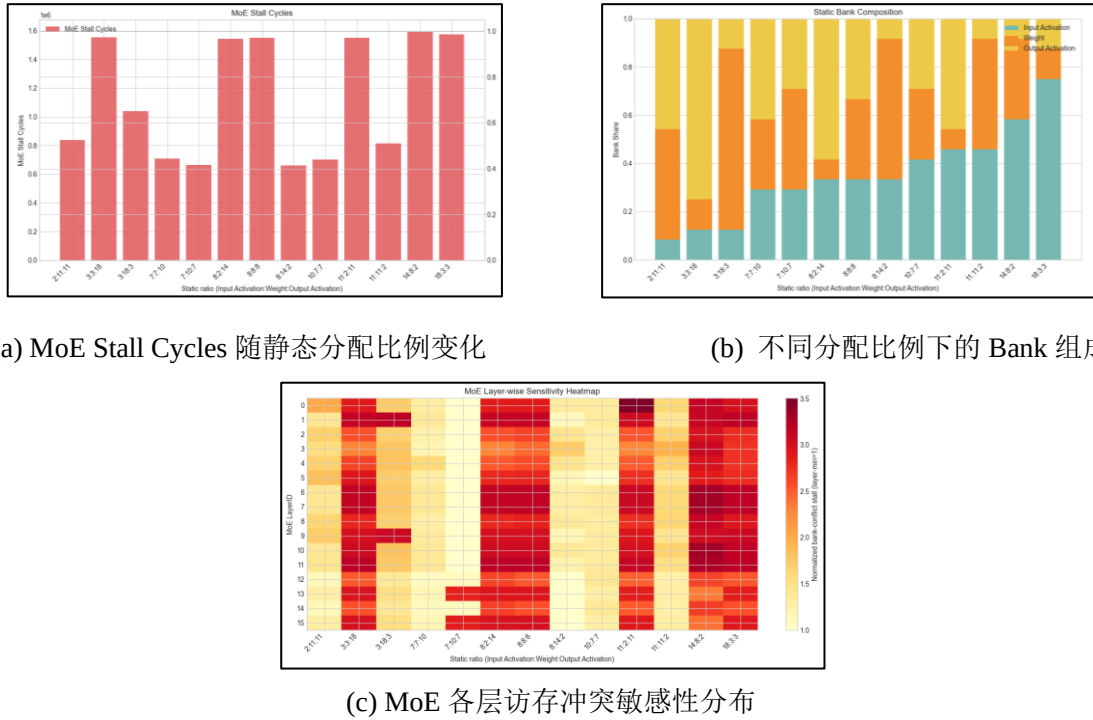


图 2-4 不同静态 Bank 分配策略对 MoE 访存性能的影响

2.3 本文架构优化动机

以上实验证明，MoE 模型在运行时存在明显的 Memory-Bound 特性，在 MoE 层遇到的问题主要是由于 Bank 冲突以及资源分配不合理造成的。传统的固定 Bank 分配不能满足各种不同的专家以及不同的网络层对内存的需求，造成某些 Bank 负担过重而另外一些 Bank 闲置，产生大量的 Stall 开销，因此需要从片上存储的角度考虑改进方法来解决这个问题。

根据以上讨论，本文针对 MoE 场景下不规则访存冲突问题提出了总线统一及存储资源

虚拟化的 MemDomain 方案，在一方面简化由于多路访问而导致的仲裁、调度等工作的同时，另一方面利用虚拟 Bank 到物理 Bank 的映射实现存储资源按需分配以及运行时重新分配，以解决 MoE 中的不规则访存冲突的问题，在接下来的内容中会对这个方案进行详细介绍。

第三章 总线统一和资源动态虚拟化的方案与实现

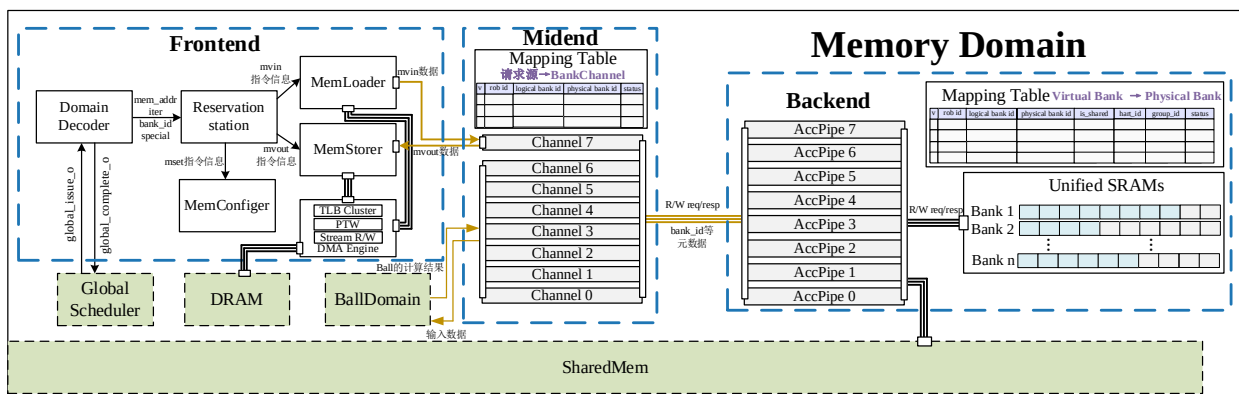
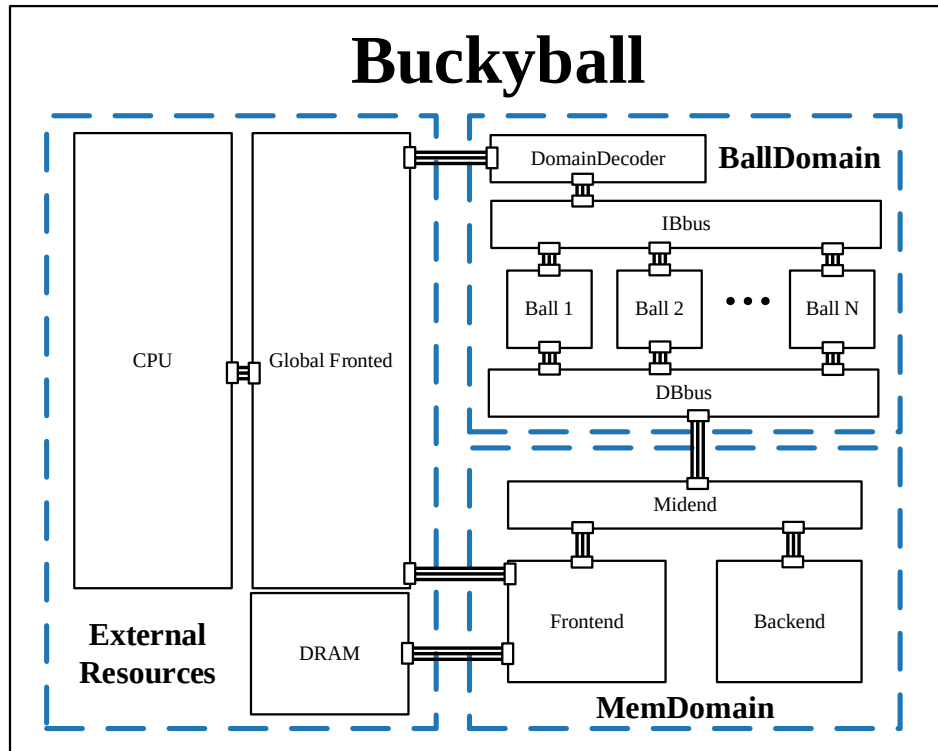
3.1 基于 Buckyball 平台的 MemDomain 总体架构设计

本文基于课题组已有的 Buckyball^[26]架构开展片上存储系统优化设计。Buckyball 是一个面向领域特定架构（Domain-Specific Architecture, DSA）的开源计算平台，在 RISC-V 基础上进行了扩展，可为硬件设计、仿真验证和软件工具链提供支撑。其整体架构采用模块化、分层化设计，主要包括 Frontend、BallDomain 和 MemDomain 等部分，如图 3-1 所示。其中，Frontend 负责前端任务分发和命令处理，BallDomain 负责计算任务执行，MemDomain 负责片上存储管理、数据搬运以及与片外存储之间的数据交互。本文并不对 Buckyball 整体架构进行重新设计，而是基于其已有计算与存储分离的组织方式，聚焦于 MemDomain 部分，面向 MoE 推理过程中的不规则访存和 Bank 冲突问题，对其访存通路、Bank 管理方式和资源调度机制进行优化。

传统 NPU 片上存储结构中，Accumulator 和 SRAM 通常具有相互独立的数据通路：Accumulator 主要用于累加结果写回，SRAM 则用于一般数据读写。该结构虽然实现简单，但会引入额外的数据通路、仲裁逻辑和控制开销。同时，在片上存储资源固定划分的情况下，不同类型数据对 Bank 资源的占用容易出现不均衡，进而导致局部 Bank 访问集中和访存冲突。对于 MoE 推理而言，由于专家激活和 Token 分布具有动态性，固定 Bank 分配方式难以适应不同层和不同专家之间变化的访存需求，因此更容易形成 Memory-Bound 问题。

因此，本文提出一种基于统一总线和存储资源虚拟化的 MemDomain 优化结构。该结构将 Accumulator 与 SRAM 的访问路径进行统一，使普通访存请求和累加写回请求能够共享同一套后端 SRAM Bank 资源；同时引入虚拟 Bank 到物理 Bank 的动态映射机制，使片上存储资源能够根据运行时访存需求进行按需分配。通过上述设计，本文希望在不改变 Buckyball 整体计算框架的前提下，提高片上存储资源利用率，缓解 MoE 模型中由动态访存引起的 Bank 冲突问题。

如图 3-2 所示，MemDomain 采用前端（Frontend）、中端（Midend）和后端（Backend）三级结构。前端与全局调度器之间通过 global_issue_i 和 global_complete_o 进行指令发射与完成反馈；BallDomain 和前端均以统一的 BankRead/BankWrite 形式接入 Midend，其中 BallDomain 请求位于前若干个入口，前端的 DMA 请求位于最后一个入口。Midend 将多源读写请求统一调度到后端 mem_req 通道，并携带虚拟地址、共享标志等元信息。后端根据



读搬运请求和片外写回请求。整体上，前端可划分为指令解码、缓冲调度和执行提交三个阶段：首先，前端从全局调度器接收内存域指令，并在 MemDomainDecoder 中解析指令类型及其操作字段；随后，MemReservationStation 对解码后的指令进行缓存和分发；最后，MemLoader、MemStorer 和 MemConfiger 分别完成数据搬入、数据搬出和资源配置操作，并将完成信息返回给全局调度器。

在命令通路上，前端主要由 MemDomainDecoder、MemReservationStation、MemLoader、MemStorer 和 MemConfiger 组成。MemDomainDecoder 根据 funct 字段识别 mset、mvin 和 mvout 三类指令，并解析出 mem_addr、iter、bank_id 和 special 等字段。解码后的指令进入 MemReservationStation，该模块采用 FIFO 结构缓存待执行指令，并根据指令类型将其分别发射到 MemLoader、MemStorer 和 MemConfiger。

其中，MemLoader 负责处理 mvin 指令，对应“片外读、片上写”的数据搬运过程。它通过 StreamReader 从片外存储读取数据，并经统一 BankWrite 接口写入片上 Bank。MemStorer 负责处理 mvout 指令，对应“片上读、片外写”的数据搬运过程。它使用 BankRead 接口从片上 Bank 读取数据并用 StreamWriter 写入片外存储中，MemConfiger 负责处理 mset 命令，根据该命令提供的 bank_id 以及 alloc 和分组信息向后端请求虚拟 Bank 分配或者释放操作，这三个执行模块结束后都以 cmdResp 形式返回各自结果给完成仲裁器，再由完成仲裁器汇总后反馈给全局调度器。

从数据流来看，mvin 以及 mvout 的访存过程如图 3-4 所示，在 mvin 中，首先由前端得到目标虚拟 Bank 的 group_count，然后由 StreamReader 读取片外的数据并由 BankWrite 通路写入片内存储；对于 mvout，前端先通过 BankRead 通路读取片上数据，再由 StreamWriter 写回片外存储。该过程中，前端并不直接感知底层物理 Bank 的具体位置，而是通过虚拟 Bank 编号和分组信息向中端、后端发起统一访问请求。

MemDomain 指令的整体执行阶段如图 3-5 所示，可划分为 alloc、issue 和 complete 三个阶段。在 alloc 阶段，全局调度器发出一条命令，前端进行解码并将该命令存放到保留站中，在 issue 阶段，保留站根据这个命令类型把 mvin、mvout 以及 mset 发送到相应的执行单元，在 complete 阶段，执行单元执行完毕数据传输或者设置之后返回一个 cmdResp 给前端，然后前端产生一个 global_complete_o 信号告知全局调度器这条命令已经完成。这样就把命令控制流水线和数据传输流水线分开并且用同样的方法对所有不同内存域的操作进行处理。

总体上，MemDomain 前端采用一个统一指令入口完成配置、数据搬入以及数据写回等操作，便于之后中端统一调度以及后端动态映射使用。

3.3 面向多源访存请求的中端统一接入与通道绑定机制

MemDomain 中端位于前端请求生成与后端存储管理之间，主要负责多源访存请求的统一接入、BankChannel 分配与请求路由。如图 3-2 中端部分所示，中端同时接收来自 BallDomain 的 BankRead/BankWrite 请求以及来自前端 DMA 的 BankRead/BankWrite 请求，并将这些请求转换为统一的后端访存接口。通过这一设计，前端数据搬运请求和 BallDomain 计算访存请求可以共享同一组后端访问通道，从而屏蔽不同请求源之间的接口差异。

需要说明的是，中端内部的映射表并不是虚拟 Bank 到物理 Bank 的映射表，而是“请求源到 BankChannel”的通道绑定表。每个表项表示当前 BankChannel 是否可用、是读请求还是写请求，还有这个 Channel 对应请求源 ID。真实的虚拟 Bank 到物理 Bank 资源分配是在后端实现，在这种层次划分上，中间层解决多个请求源到有限后端 Channel 分配的问题，而后端解决虚拟 Bank 到实际 SRAM Bank 资源分配问题。

在请求分配阶段，若有请求源发出请求并且未与 BankChannel 关联，则中端会选择一个空闲 BankChannel 并将其与之关联，在通道映射表中记录这种联系。完成之后，该请求源之后的所有请求以及相应消息都会经过这个 BankChannel 传输到后端，并附带 bank_id、group_id 等附加信息。bank_id 是本次请求所要访问的虚拟 Bank 编号，在多个 Bank 映射情况下使用 group_id 来标识不同的物理 Bank 组。

在图 3-4 所示的数据流中，mvin 和 mvout 均需要经过中端完成 BankChannel 分配。对于 mvin，中端接收来自前端 BankWrite 的写入请求，并将其转发到后端；对于 mvout，中端接收来自前端 BankRead 的读取请求，并将其转发到后端。也就是说，中端不区分请求来源于计算单元还是 DMA 搬运模块，而是统一按照后端 BankChannel 访问语义进行调度。

对于读请求，在有足够空闲通道的情况下，中端可以为多个请求源 BankChannel 服务，增加读并行量；而对于写请求，则为了避免多个写请求同时对后端产生竞争导致控制复杂度过高，在每个时钟周期中，中端只会给一个需要分配的写请求分配一个 BankChannel。当这个请求完成之后，如果某个 BankChannel 在接下来一段时间内都没有新请求或者没有新响应发生，那么就将其对应的映射表项清空，这个 BankChannel 就又回到了空闲状态，

可以供其他请求使用。

总体而言，中端利用请求源到 BankChannel 动态绑定的方式，实现多个请求源共用少量后端通道的需求，在此过程中也保持了前端、BallDomain 以及后端接口语义一致性的同时也给后端动态 Bank 映射及统一 SRAM/Accumulator 访问提供稳定请求入口。

3.4 面向虚拟 Bank 映射的后端动态资源管理机制

MemDomain 后端负责接收来自中端 BankChannel 的统一读写请求，并完成虚拟 Bank 到物理 Bank 的映射、访问路径选择、SRAM 读写以及响应返回等操作。如图 3-2 后端部分所示，中端将多源请求绑定到不同的 Channel 后，通过统一的 R/W req/resp 接口将请求送入后端，同时携带 bank_id、group_id 等访存元信息。后端根据这些元信息查询映射表，选择实际访问的物理 SRAM Bank。

后端主要由 AccPipe 阵列、虚拟 Bank 映射表和 Unified SRAMs 三部分组成。其中，AccPipe 阵列对应多个后端访问通道，负责承接来自中端的读写请求，并在普通 SRAM 访问和累加访问之间进行统一处理。虚拟 Bank 映射表用于记录上层可见的逻辑 Bank 与底层物理 Bank 之间的对应关系，其表项主要包括虚拟 Bank 编号、物理 Bank 编号、分组编号和状态位。Unified SRAMs 则由多个物理 SRAM Bank 组成，是最终完成数据存储和访问的实体。

映射管理是后端实现动态资源分配的核心。mset 指令驱动的虚拟 Bank 动态映射流程如图 3-3 所示。当前端中的 MemConfig 根据 mset 指令产生配置请求后，后端根据配置请求中的虚拟 Bank 编号、分配/释放标志和分组信息更新映射表。对于分配操作，后端从当前空闲的物理 Bank 中选择可用项，并在映射表中写入虚拟 Bank 到物理 Bank 的映射关系；对于释放操作，后端清除该虚拟 Bank 对应的映射表项，使相关物理 Bank 重新回到可分配状态。通过这种方式，上层模块只需要使用虚拟 Bank 编号进行访问，而不需要关心底层物理 Bank 的具体位置。

当 mvin 或 mvout 访问某个虚拟 Bank 时，其基于映射表的访存数据流如图 3-4 所示。前端首先获取该虚拟 Bank 对应 group_count。对于单 Bank 映射，请求只对应一个物理 Bank；对于多 Bank 映射，MemLoader 或 MemStorer 会在数据搬运过程中维护 group_counter，使连续数据块按照不同 group_id 分散到多个物理 Bank 中。后端收到来自中端的访问请求后，根据请求中的 bank_id 和 group_id 查询映射表：当表项中的虚拟 Bank 编号与请求的 bank_id

一致，并且分组编号匹配时，该表项对应的物理 Bank 被选中，随后请求进入对应的 AccPipe 和 SramBank 完成访问。

AccPipe 是后端统一普通 SRAM 访问与 Accumulator 访问的关键模块。对于普通读请求，AccPipe 将请求转发至对应的物理 SRAM Bank，并将读响应沿原通道返回；对于普通写请求，当 wmode=0 时，写数据直接写入 SRAM Bank；对于累加写请求，当 wmode=1 时，AccPipe 先从 SRAM Bank 中读取旧值，再将旧值与输入数据进行累加，最后写回同一地址。这样，原本需要独立 Accumulator 数据通路完成的“读一改一写”操作被合并到统一 SRAM 访问路径中，实现了 Accumulator 与 SRAM 访问接口的统一。

总体来看，后端通过“虚拟 Bank 映射表+AccPipe 阵列+Unified SRAMs”的结构，将上层的逻辑 Bank 访问转换为底层的物理 Bank 访问。一方面，虚拟 Bank 映射机制使片上存储资源能够根据运行时需求动态分配；另一方面，统一 AccPipe 访问路径使普通存储访问和累加访问共享同一套 SRAM Bank 资源。该设计为 MoE 场景下缓解 Bank 冲突、提高片上存储利用率和支持多 Bank 并行访问提供了硬件基础。

3.5 面向软硬件协同调度的自定义访存指令设计

为了支持 MemDomain 对数据进行显式搬移以及对 Bank 资源进行动态管理，本文引入三类自定义指令：mset、mvin 和 mvout。其中，mset 用于虚拟 Bank 的分配、释放以及多 Bank 分组配置；mvin 用于将片外存储中的数据搬运到片上 Bank；mvout 用于将片上 Bank 中的数据写回片外存储。三类指令分别对应图 3-3 和图 3-4 中的配置流与数据流。

mset 指令是动态资源分配机制的入口，其执行过程如图 3-3 所示。前端解码 mset 后，将其发射至 MemConfiger。MemConfiger 根据 special[9:5]判断是否需要为同一个虚 Bank 分配多个物理 Bank。当 group_count=1 时，仅生成一次配置请求；当 group_count>1 时，MemConfiger 依次生成多个配置请求，并为每个请求分配不同的 group_id。后端收到配置请求后选择空闲物理 Bank，并将虚拟 Bank、物理 Bank 和分组编号写入映射表，从而完成虚拟 Bank 到物理 Bank 的动态绑定。

mvin 和 mvout 是数据搬运指令，其执行过程如图 3-4 所示。二者访问的对象均为虚拟 Bank，而不是固定物理 Bank。对于 mvin，MemLoader 先获取目标虚拟 Bank 的 group_count，随后将片外读取的数据按照 group_id 写入对应物理 Bank；对于 mvout，MemStorer 同样根据 group_count 和 group_id 从对应物理 Bank 读取数据，再通过 StreamWriter 写回片外存储。

通过这种方式，软件层只需面向虚拟 Bank 编程，具体物理 Bank 的选择和数据分散由硬件完成。

从执行阶段看，三类指令均遵循图 3-5 所示的 alloc—issue—complete 流程。alloc 阶段完成指令接收、解码和保留站入队；在 issue 阶段根据不同指令类型发送至 MemLoader、MemStorer 或者 MemConfiger，在 complete 阶段由相应处理单元给出 cmdResp 并通过全局完成接口提供该命令的 rob_id。所以自定义指令不仅有软件可见的 Bank 管理以及数据搬运的方法，而且也符合硬件内部的一致性调度、动态映射以及完成反馈等机制。

如表 3.1 所示，mset、mvin 和 mvout 采用相同的寄存器编码方式。其中，bank_id 表示软件可见的虚拟 Bank 编号，在 mvin 和 mvout 中，mem_addr、depth/iter 和 stride 用来指定片外的数据搬运范围，在 mset 中，alloc 和 group_count 来管理虚拟 Bank 资源的分配、回收及多 Bank 映射等操作，这样可以保证 MemDomain 在简化其软件接口的同时，又能给后面的 Bank 映射提供必要的硬件相关信息。

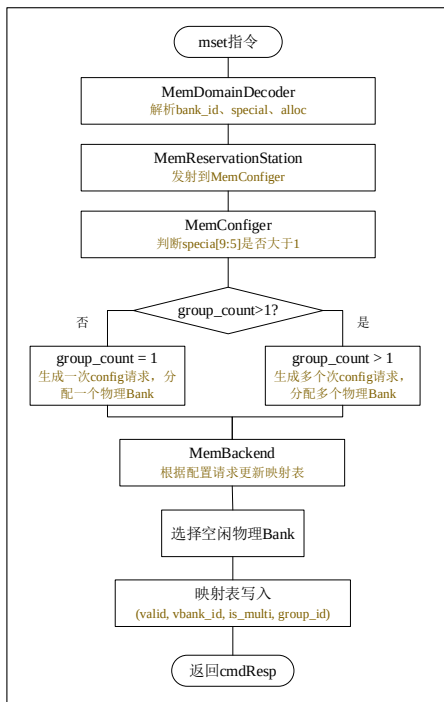


图 3-3 动态映射流程

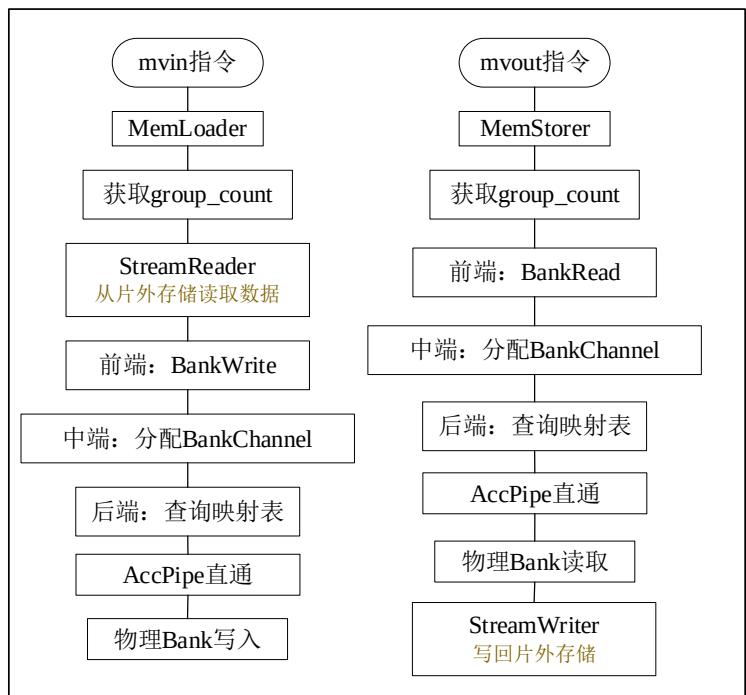


图 3-4 mvin/mvout 基于映射表的访存数据流

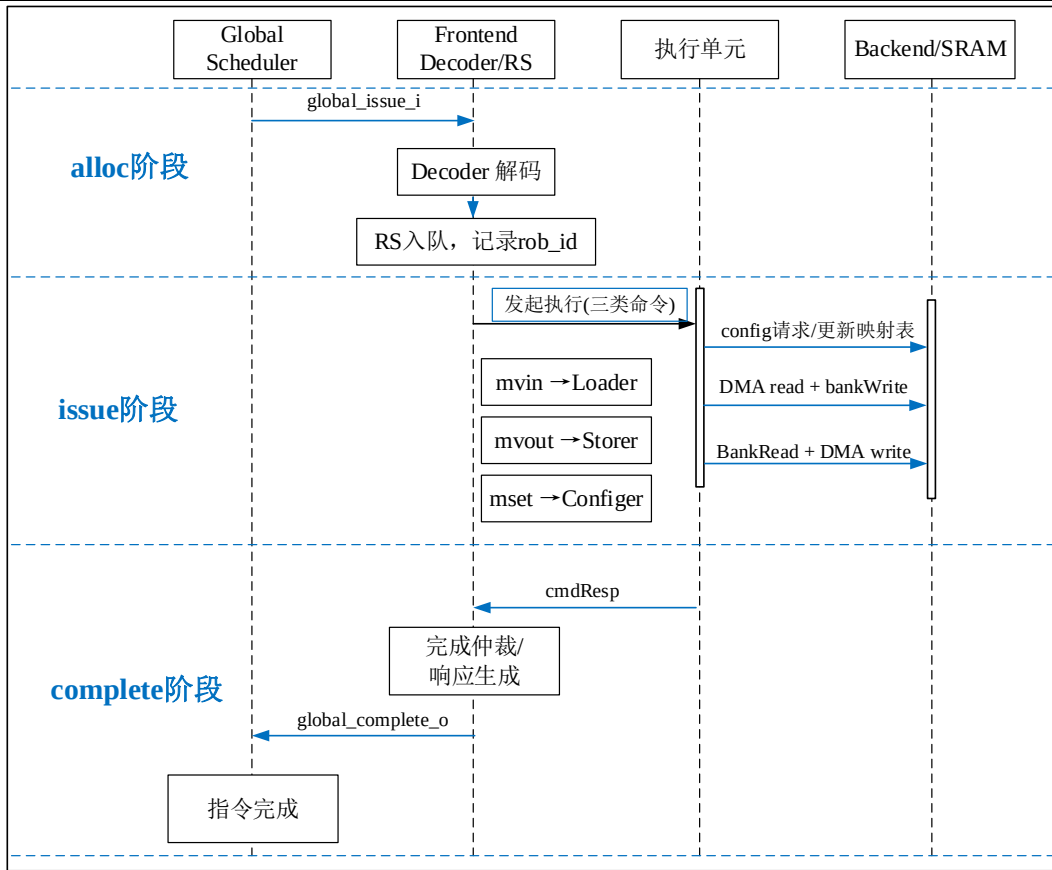


图 3-5 MemDomain 指令执行阶段时序

表 3.1 mset mvin mvout 指令字段表

mset	字段位置	作用
bank_id	rs1[9:0]	虚拟 Bank 编号
row	rs2[4:0]	待用
col	rs2[9:5]	多 Bank 分配数量；大于 1 时表示该虚拟 Bank 映射到多个物理 Bank
allocator	rs2[10]	1=分配/0=释放
reserved	其他	待用

续表 3.1 mset mvin mvout 指令字段表

mvin	字段位置	作用
bank_id	rs1[9:0]	虚拟 Bank 编号
depth	rs1[63:30]	迭代次数
mem_addr	rs2[38:0]	内存地址
stride	rs2[57:39]	跳步传输
reserved	其他	待用

mvout	字段位置	作用
bank_id	rs1[9:0]	虚拟 Bank 编号
depth	rs1[63:30]	迭代次数
mem_addr	rs2[38:0]	内存地址
stride	rs2[57:39]	跳步传输
reserved	其他	待用

3.6 预取与动态 Bank 分配的协同优化机制

MoE 推理过程中,不同专家的激活情况和 Token 分布具有明显动态性,导致输入激活、专家权重和中间结果在不同时间段对片上 Bank 资源的需求不同。如果仅在数据被计算单元实际请求时再进行搬运,容易造成计算单元等待数据,从而增加访存延迟。因此,本文在统一总线和虚拟 Bank 映射机制的基础上,引入面向片上存储资源的预取协同机制,使数据搬运能够与计算执行过程部分重叠。

本文预取机制的设计目标包括两点:一是在数据被正式访问前,提前完成片外数据到片上 Bank 的搬运,从而降低计算阶段的等待时间;二是在预取数据写入片上 SRAM 之前,利用动态 Bank 分配为它选取一个合理物理 Bank,防止预取增加 Bank 冲突。

预取触发条件是基于指令流以及资源状态而定。当更低一层或者下一个专家数据搬运请求已到达前端保留站并且其需要访问的目标虚拟 Bank 还未准备好时,系统就可以进行 mset 及 mvin 操作。即先由 mset 为该数据分配相应数量的物理 Bank 并创建虚拟 Bank 与物

理 Bank 之间映射；再由 `mvin` 依照这个映射关系把片外数据加载到相应的物理 Bank 上。如果目标 Bank 资源不够或者当前访存通道很忙，则可以推迟这个预取请求以免影响正在进行中的计算访存操作。

本文采用固定预取窗口的方式控制预取范围。预取窗口 W 表示提前准备未来 W 个数据块或专家计算单元所需的数据。当 $W=0$ 时，不进行提前预取，数据在实际使用时才搬入；当 $W=1$ 时，系统提前准备下一数据块或下一专家所需数据。较大的预取窗口可以增加计算与访存重叠机会，但也可能提前占用过多片上 Bank 资源，因此需要与动态 Bank 分配机制配合使用。

本文预取对象主要包括下一计算阶段所需的输入激活和专家权重数据，而输出结果不作为预取对象。而对于 MoE 层来说，预取是以专家为单位进行的，在当前专家计算时，就为下一个专家准备好 Bank 并把数据搬进去。如果是有多 Bank 映射的情况，则预取的数据会根据 `group_id` 分散地写入不同的物理 Bank 中，从而避免过多的数据集中在某一个 Bank 上导致的争用问题。

预取机制与动态 Bank 分配相互配合，在传统的固定 Bank 分配时，预取的数据只能写入事先划分好的 Bank 区域，易造成与其他正在使用的数据发生资源的竞争，在本设计中，预取之前可以先用 `mset` 获取一个空闲的物理 Bank，然后利用虚拟 Bank 映射表来决定最终写入的位置，因此，预取的操作既可以准备好要读取的数据，也可以依据当前 Bank 的情况选择最优的位置进行存储，从而减少预取对正常的访存造成的影响。

第四章 实验验证

4.1 总线统一与存储资源虚拟化的正确性验证

为了验证所提出的 MemDomain 架构的有效性，本节基于 Buckyball 平台中的 BBtest 测试工具开展软硬件协同功能验证，主要从统一总线接口语义、存储资源虚拟化以及动态 Bank 分配三个方面进行测试。该实验采用 RTL 级硬件，使用 16×16 整数矩阵乘法作为基础测试用例，输入位宽均为 INT8，输出位宽为 INT32。测试中片上存储大小、Bank 带宽及数据流等相关信息均参照第二章表 2.1 中的配置，程序由统一命令行依次完成资源设定（mset）、数据搬运（mvin）、矩阵运算以及输出结果（mvout）等工作之后再得结果与 CPU 端基准程序逐位比较来判断是否正确完成运算及内存管理任务。当然这里仅选取 16×16 矩阵乘法并不是用来涵盖所有网络规模，更多的是为了检验文中所提出的方法对于 MemDomain 的关键功能路径即数据搬运、虚拟 Bank 分配、动态映射、多 Bank 写入和结果搬运等功能是否有效支持。

从实现上，输入的操作数分别对应不同的虚拟地址（vbank0、vbank1），而计算得到的结果保存在一个虚拟地址 vbank2 中，在累加时为 vbank2 分配多个物理地址以并行地写回从而提高访存带宽利用率。其主要代码如图 4-1（a）所示。

从执行角度上看，统一总线接口下任务执行具有明显的阶段性。由执行日志可以看到每一条指令都是从资源分配（alloc）、指令发射（issue）到最后执行完成（complete）的过程，形成一个循环。为了进一步确认这个执行机制是否正确，如表 4.1 所示是几个代表性指令执行的日志信息。可以看出对于同一个指令（rob_id=5），alloc、issue、complete 这三个事件按时间先后顺序发生并且它们的功能是一样的。这也说明统一接口对指令调度及执行有相同的意义表示。也就是说所设计的统一总线接口可以在各个不同的执行阶段之间进行合理的通信，从而实现跨模块的协调工作。

对于存储访问行为，波形图还显示了虚拟存储体到物理存储体之间的转换过程，如图 4-2（a）所示，在输入过程中，用于进行矩阵乘法运算的两个输入矩阵分别对应虚拟地址 VA0 以及 VA1 并被映射到物理地址 PA0 以及 PA1，说明输入数据是一一地存储；而在图 4-2（b）中则显示了输出矩阵所对应的存储分配情况，因为矩阵乘法操作之后会有比特膨胀，所以输出的数据量更大，因此 MemDomain 为虚拟地址 VA2 分配了 PA2 至 PA5 共 4 个物理 Bank，从波形可以观察到不同的 Channel 上的资源分配以及数据写入并不是同时发生的，而是在连续几个时钟周期里依次完成，这是由于 RTL 内部握手、仲裁及寄存器锁存导致流水线化执行的结果。

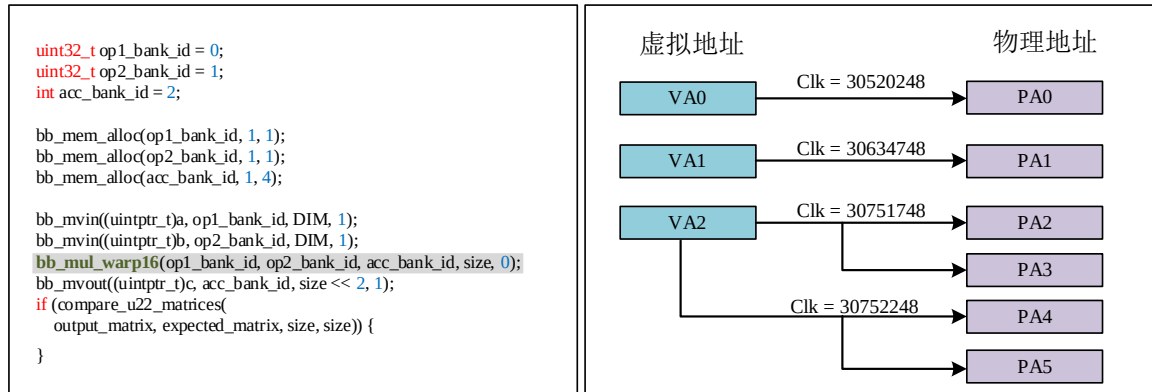
进一步地，日志信息从事务级的角度对上述映射关系进行了验证。需要注意的是，波形图表示的是硬件内部每个时钟周期的变化情况，而日志则是更高层面上的传输事务或者批次事件，所以两者的时间粒度并不是完全相同的，在日志中，同一批次中的数据传输可能会被记录成同一个时钟周期，在波形中，同一批次中不同的通道的请求也有可能会表现为相差一个时钟周期的流水线式推进。而且目前矩阵乘法运算符一次只生成两个通道的数据，因此 VA2 到 PA2-PA5 的写入操作将会被分成两部分来进行：前面两个物理 Bank 收到第一部分输出数据，而后面两个物理 Bank 则要等到计算端再给出下一组输出之后才能开始写入，所以在日志中可以看到每两个通道的导入时间一样，但是另外两个通道比之前一批滞后几百个时钟周期的情况。

从性能上看，性能监测计数器（Performance Monitoring Counter, PMC）统计结果表明矩阵乘计算部分耗时为 61 个时钟周期，而数据搬运以及回写部分耗时为 4168 个时钟周期。这意味着，在当前测试条件下，片上数据读取以及传输所花费的时间较长，呈现典型的 Memory-Bound 特性。另外，如图 4-1（b）、图 4-2（b）所示，输出矩阵对应虚 Bank VA2 被绑定到 PA2、PA3、PA4、PA5 这四个物理 Bank 上以实现累加结果向多个物理 Bank 写入的目的。而在 RTL 仿真日志中对所有 vbank_id=2 的写入操作计数，可得到共有 64 次写入。而在动态多 Bank 情况下，这 64 次写入实际上只占用了 34 个时钟周期，即平均每 3.4 个时钟周期就有一个写入发生；但如果是所有累加结果都固定写入一个物理 Bank，则由于单个物理 Bank 在一个时钟周期内不能同时处理多个写入请求，所以这 64 次写入至少需要 64 个时钟周期才能够完成。因此仅从累加写入这一方面考虑，动态多 Bank 分配可以使写入完成所需的时间从理论上不低于 64 个时钟周期缩短到不超过 34 个时钟周期，即节省了至少 30 个时钟周期，相当于降低了约 46.9% 的写入排队延迟。由此可见，动态多 Bank 映射不仅可以准确地实现虚 Bank 到实 Bank 的映射关系，而且还可以在累加写入期间增加片上存储访问的并发性，从而避免由于单个物理 Bank 写入过多导致的排队现象。同时根据仿真波形也可以观察到，本设计中的统一总线及动态资源分配策略确实可以在保证正确性的前提下有效地解决局部 Bank 间的冲突问题。

实验结果从功能以及行为上对上述设计进行了测试：首先是统一总线接口上的执行语义是正确的并且稳定的；其次，虚拟地址到物理地址的动态映射能够在运行时生效并且可以支持多个 Bank 同时读写操作；最后，硬件计算结果与 CPU 参考结果相同，说明计算正确性以及内存管理的一致性得到保证。

表 4.1 指令执行阶段日志示例

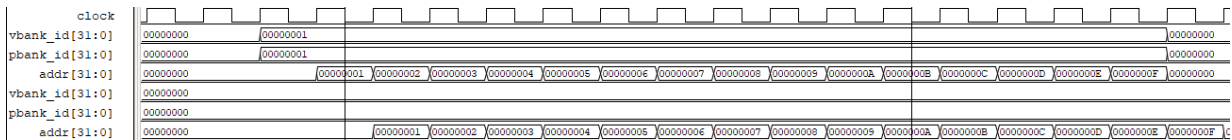
时钟周期	事件类型	rob_id	funct	执行阶段说明
30736248	alloc	5	0x40	资源分配阶段
30736748	issue	5	0x40	指令发射阶段
30767748	complete	5	0x40	执行完成阶段



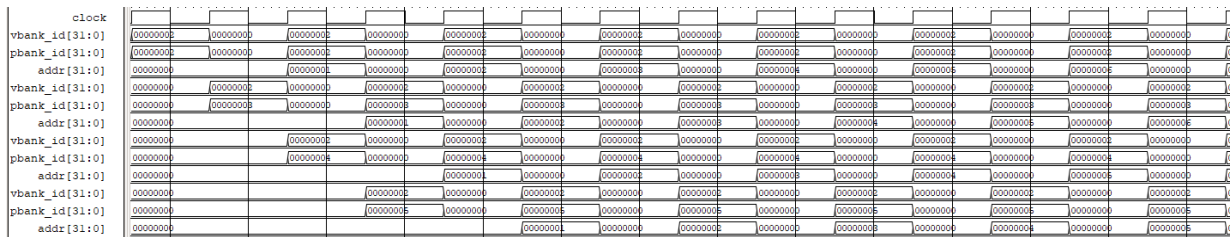
(a) 矩阵乘计算流程及关键代码

(b) 虚拟地址到物理地址的动态映射

图 4-1 统一总线架构下的计算流程与动态地址映射机制



(a) 输入矩阵 VA0/VA1 到 PA0/PA1 的波形验证



(b) 输出矩阵 VA2 到 PA2 - PA5 的波形验证

图 4-2 功能仿真波形验证图

4.2 动态资源分配机制的优化验证

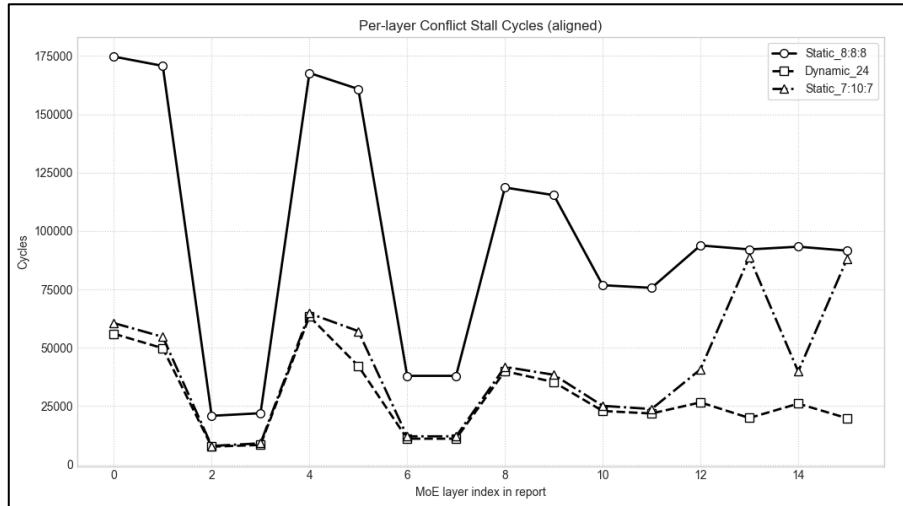
为了验证所提出的动态资源分配方案的有效性，在两个维度上进行实验研究：一是固定资源情况下比较不同的资源分配方式的效果；二是不同规模资源的情况下考察动态资源分配的有效性和灵活性。我们仍然使用 MoDSE 网络模型进行研究。需要注意的是，虽然

本文提出的优化是针对新型片上存储设计，但是直接在 RTL 级进行大规模仿真实验成本较高，因为 MoE 网络层数较多、访存行为较复杂，整个仿真过程耗时较长，无法在合理的时间内进行大量的参数变化和比较。所以这里使用周期级的模拟器 SCALE-Sim 来模拟和评价片上存储的访问行为，这个模拟器可以保持与目标架构相同的 Bank 数量、带宽限制以及冲突检测的方式，主要关注 Bank 冲突、资源分配以及访存调度等方面的行为，因此可以较好地反映出不同的资源分配方式之间的优劣关系。

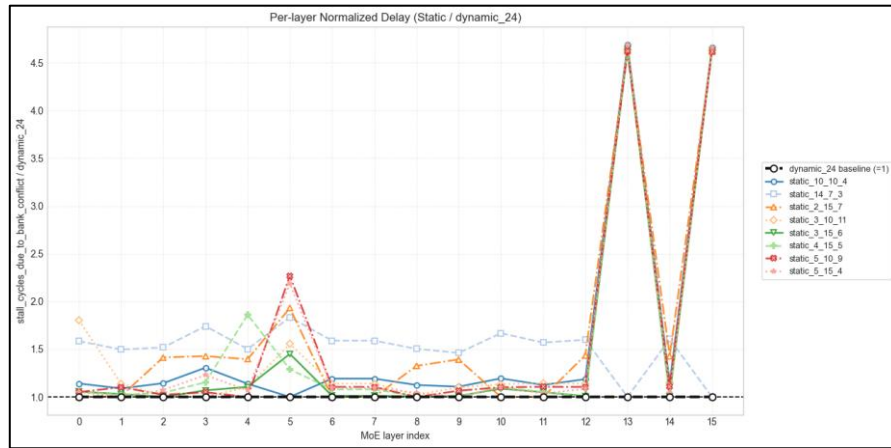
固定资源条件下的分配策略比较：首先，在固定 Bank 带宽以及总 Bank 数目相同情况下，比较这几种不同 Bank 的分配方法对于每一个 MoE 层的效果。需要说明的是，本文并未对所有可能的静态 Bank 分配组合进行完全穷举，而是根据资源均衡性和前期粗粒度搜索结果，选取 Static_8:8:8 作为均衡静态基准，选取 Static_7:10:7 作为本文测试范围内表现较好的代表性静态配置，用于与动态分配策略进行对比。有两种静态分配的方法 Static_8:8:8、Static_7:10:7 和一种动态分配的方法 Dynamic_24。这里说明一下，Static_x:y:z 指的是使用静态方案，三个矩阵（Input Activation、Weight 与 Output Activation）的分配数量分别为 x、y、z；Dynamic_x 指的是使用动态方案，总资源是 x 个 Bank。我们计算每一层因为 Bank 冲突而造成的 Stall Cycles，如图 4-3（a），归一化的延迟如图 4-3（b），动态分配的具体 Bank 分配情况如表 4.2。

由图 4-3（a）可知，在各个 MoE 层上，Dynamic_24 具有较少 Stall Cycles 并且在不同层间波动较小，表明其具有更好的内存访问行为；而两种静态方案中，Static_7:10:7 显著优于 Static_8:8:8，即通过合理地对 Bank 进行分配，可以在一定程度上解决内存竞争问题；但是 Static_7:10:7 也无法在所有 MoE 层上都占据主导地位，在后面的 MoE 层上，它所消耗的时间比 Dynamic_24 大很多。这说明静态分配虽然可以在部分 MoE 层上获得良好结果，但是对于不同的 MoE 层有不同的内存要求时，是不能满足所有 MoE 层的需求的。

层间访存需求变化造成静态分配失调：如图 4-3（b）所示，静态分配的不足之处在于其受到层间访存需求的影响较大。对某一层次来说，某种 Bank 分配比例可以很好地满足这个层次的需求，从而使它接近动态分配的效果；但是当神经网络运行时，各个层次的 Input Activation、Weight 以及 Output Activation 的访存量不断发生变化，原来较为合适的静态比例已经不能适应当前的访存情况，于是会出现一些 Bank 被闲置在需求较少的地方，而在需要较多的地方又发生访存集中现象以及排队的现象。动态分配的优势并不是某一个固定的比例值，而是可以根据不同的层次访存情况进行调整，以此来解决层间访存需求变化造成的静态分配失败的问题。



(a) 不同分配策略在各 MoE 层上的 Stall Cycles 对比



(b) 不同分配策略在各 MoE 层上的归一化延迟对比

图 4-3 不同分配策略在各 MoE 层上的性能对比

Bank 利用率角度的性能差异比较：为进一步分析导致 Static_7:10:7 与 Dynamic_24 在不同层段上具有不同性能的原因，在 MoE 层中将其分为前半段和后半段并计算 Input Activation、Weight 和 Output Activation 这三种类型的数据在每个段上的平均 Bank 利用率以及平均利用不均衡程度，如表 4.3 所示，E0-E4 指的是前四个 MoE 专家对应的网络。这里，平均利用不均衡程度用来表示这三种类型的资源分配情况，数值越大，则 Bank 资源利用就越不平衡。

从表 4.3 可以看出，在 MoE 层前四个专家中，Static_7:10:7 与 Dynamic_24 的效果相差不多，说明这个固定的比率对于这部分层来说可以很好地满足其内存访问的需求。但是从 Bank 利用率上看，Static_7:10:7 仍然有比较明显的不平衡，其中 Weight 利用率远大于 Input Activation 以及 Output Activation，即 Weight 承受的压力较大。而相比之下，Dynamic_24 这三种类型的数据利用率都比较均匀，平均不均衡度较低，因此可以看出动态划分可以根

据每层内存访问的特点来合理分配资源的比例。在后半部分 MoE 层当中，Static_7:10:7 和 Dynamic_24 之间的差别较大，在利用率上，Static_7:10:7 的 Weight 侧利用率仍处于较高水平，而 Input Activation 和 Output Activation 的利用率较小，即由于固定的分配比例使得资源进一步偏向 Weight 侧造成局部热点问题；另外后半部分层的平均利用不均衡程度也有所上升，说明静态的方式已不能很好地满足它们的需求。但是 Dynamic_24 可以根据实际情况改变三类数据的 Bank 分配比例，使得资源利用更加符合每层实际需要，所以在后半部分层中的平均延迟较低。

需要指出的是，为某一种类型的数据分配一定数量的 Bank，并不代表能够获得相应的真正有效的并行度。对于有一定间隔或者周期性的访存请求，如果其地址映射关系以及 Bank 数量之间有不良搭配，则该类请求可能会集中在少数几个物理 Bank 上，造成实际上参与处理的 Bank 小于被分配的数量。所以 Bank 数量增多也不一定能使访存延迟一直减少。Static_7:10:7 虽然为 Weight 分配了很多 Bank，但是由于其映射关系是固定的，在某些层中，Weight 还会有访存热点。而 Dynamic_24 的优势并不是单纯地增加某种类型的 Bank 数量，而是根据不同层的访存特性来重新分配 Input Activation、Weight 和 Output Activation 所占有的 Bank 数量和它们之间的映射方式以达到更高的实际 Bank 并发性和更低的局部访存冲突。

不同 Bank 数量下的稳定性和可扩展性比较：在此基础上，本文还研究了不同的资源量对于所提出的方法的影响。在固定其他所有硬件参数的情况下，改变 Bank 数量，比较动态分配和静态分配方法的整体效果的变化情况，如图 4-4 所示。由图可见，在各种 Bank 数量上，动态分配相比于相同资源情况下的静态分配整体上更好或者至少一样好，在大多数情况下能获得更低总的延迟以及冲突开销，说明这种方法的效果具有鲁棒性，而不是仅仅在某个特定资源分配时有效。

同时也可以看出，当 Bank 数量增多时，动态分配的优势更为明显。这是因为有更多的资源可供使用，在这种情况下，动态分配可以更好地发挥出所有可利用 Bank 的作用，做到更优的访存调度；而由于静态分配资源是固定的，无法根据每层之间不同的请求来改变，所以它不能一直有好的表现。另外从冲突延迟上来看，在大多数情况下，动态分配都可以有效地减少由于 Bank 冲突导致 Stall Cycles，也说明了它对于缓解冲突是有帮助的。

动态资源分配方案的性能结论总结：在相同的硬件资源情况下，Bank 分配方式对于系统的性能有着较大的区别。由实验可以看出，本文提出的一种动态资源分配方法可以根据运行时不同的访存需求进行相应的调整，在每一层 MoE 中不断地减少由于 Bank 冲突导致的 Stall 开销，相比于固定的分配方式有更好的效果以及更高的稳定性。并且当更多的 Bank

时，这种方法的优势更加明显，可以更好地利用片上的存储空间而不受到固定分配而导致的空间浪费的影响。所以可以说，在 MoE 的情况下，造成系统低效的原因并不是 Bank 的数量不够，而是在于资源分配不能满足实际访存的需求，而本文提出的这种动态分配方式就可以解决这个问题。

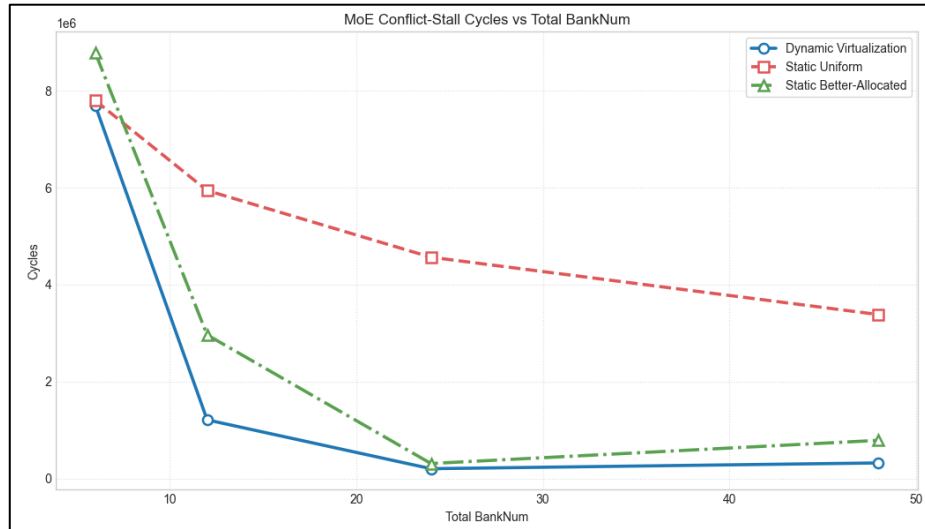


图 4-4 不同 Bank 数量下分配策略性能对比

表 4.2 动态分配策略的 Bank 资源分配方案

MoE 层数	Bank_ Input	Bank_ Weight	Bank_ Output
MoE-E0-FF1	2	15	7
MoE-E0-FF2	5	15	4
MoE-E1-FF1	3	15	6
MoE-E1-FF2	3	10	11
MoE-E2-FF1	5	10	9
MoE-E2-FF2	10	10	4
MoE-E3-FF1	2	15	7
MoE-E3-FF2	2	15	7
MoE-E4-FF1	5	10	9
MoE-E4-FF2	5	15	4
MoE-E5-FF1	2	15	7

续表 4.2 动态分配策略的 Bank 资源分配方案

MoE 层数	Bank_ Input	Bank_ Weight	Bank_ Output
MoE-E5-FF2	2	15	7
MoE-E6-FF1	4	15	5
MoE-E6-FF2	14	7	3
MoE-E7-FF1	4	15	5
MoE-E7-FF2	14	7	3

表 4.3 代表性静态方案与动态方案的 Bank 利用率对比

策略	Input Activation 平均利用率	Weight 平均利用率	Output Activation 平均利用率	不均衡度
Static_7:10:7 E0-E3	0.91	9.00	3.66	13.42
Dynamic_24 E0-E3	1.44	7.28	3.68	5.27
Static_7:10:7 E4-E7	0.59	9.00	2.36	19.97
Dynamic_24 E4-E7	1.00	8.16	2.98	16.96

4.3 预取与动态 Bank 分配协同验证

为进一步验证预取机制与动态 Bank 分配之间的协同效果，本文在相同 MoE 工作负载和硬件配置下设置预取消融实验。实验比较静态 Bank 分配和动态 Bank 分配在无预取与开启预取两种情况下的性能差异。其中，Static 表示 Input Activation、Weight 和 Output Activation 各固定分配 8 个 Bank；Dynamic 表示在总 Bank 数为 24 的条件下根据运行时访存需求进行动态分配。预取窗口 $W=0$ 表示不启用预取， $W=1$ 表示提前为下一计算阶段所需的 Input Activation 和 Weight 数据发起预取请求。实验结果如表 4.4 所示。

从表 4.4 可以看出，在静态配置下，开启预取后系统总周期由 1870007 降低至 1584955，

降低约 15.24%；有效的访存延迟从 1549983 降到 1264931,减少约 18.39%，这表明即使是在固定的 Bank 分配下，预取也可以通过提前搬移数据来掩盖一部分访存延迟给计算带来的影响。

而在不使用预取时，动态方案总周期为 781811，比静态配置减少约 58.19%，有效访存延迟从 1549983 降到 461787，下降约 70.21%，这表明动态 Bank 分配可以较好地解决由于静态 Bank 划分不能满足 MoE 动态访存而造成 Bank 冲突以及访存等待问题。

进一步地，在此基础上开启预取窗口之后，整个系统的时钟周期进一步下降到 541827，相比于之前动态方式下没有预取下降近 30.70%，有效访存延迟也下降到 221803，相对之前的动态方式下没有预取下降接近 51.97%，由此可见动态 Bank 分配与预取是相辅相成的关系：动态 Bank 分配给预取的数据提供了更多的可选物理 Bank，从而减少了预取的数据与其他正在进行运算的数据对物理 Bank 的竞争；而预取又能进一步掩盖由于动态 Bank 分配而导致的访存延迟。

总体而言，在本文所进行的四种不同配置下，Dynamic+PrefetchWindow=1 获得最小端到端执行时间和有效访存延迟。相比于静态配置无预取基准值，此方法总周期减少约 71.03%，有效访存延迟减少约 85.69%。由此可见，仅预取可以一定程度上隐藏访存等待时间，仅动态 Bank 分配可以减少 Bank 间竞争，两者结合对于 MoE 推理中的片上存储来说有较大帮助。

表 4.4 预取与动态 Bank 分配协同实验结果

分配策略	Total_Cycle	Stall_Cycle
Static/W=0	1870007	1549983
Static /W=1	1584955	1264931
Dynamic/W=0	781811	461787
Dynamic /W=1	541827	221803

4.4 MoDSE 工作负载级实验验证

在前述实验中，本文已从层级访存行为角度验证了动态资源分配机制在降低 Bank 冲突方面的有效性。进一步地，本文在 MoDSE 工作负载级别对整体执行性能进行验证。需要说明的是，本文所称“端到端”并非指从自然语言输入、Tokenizer 到 LM_Head 输出的完整

大语言模型推理流程，而是指在周期级模拟器中，从 MoDSE 工作负载专家网络的第一个仿真层到最后一个仿真层的完整执行过程，用于评估不同 Bank 分配策略、带宽配置和端口数量对整个仿真网络执行周期的影响。

实验仍然使用上文中的 MoDSE 的专家层的网络结构以及同样的输入规模下进行，比较两种不同的分配方式即动态分配策略（Dynamic_24）和静态分配策略（Static_8:8:8）。此外还从带宽和端口数量上扩大系统的资源分配。如下表 4.5 是具体的实验参数以及相应的测试结果，在这里为了方便比较，把动态分配在基准配置下的时延作为 1 来表示其他各种情况下得到的结果如图 4-5。需要说明的是本节端到端实验中的静态方案采用 Static_8:8:8 作为基础静态基线，主要用于衡量动态资源分配相对于固定均衡划分方式的整体收益；该结果不用于证明动态方案优于所有可能的静态划分组合。

动态资源分配在端到端执行过程中有良好表现：从整个实验来看，在 Baseline 情况下，动态分配总的执行周期是 781811，而静态分配需要 1870007 个周期。相比于静态分配，动态资源分配大大降低了端到端执行延迟，说明它不仅仅在局部访存访问上具有优势，在整个推理过程也能够起到很好的缓解访存压力的效果。同时结合 Stall Cycles 可以看出，在静态分配中有很多的时间是在等待访存资源，在动态分配中由于可以在运行时进行资源调度解决局部热点冲突所以减少了因为 Bank 冲突引起的流水线堵塞，因此可以看出系统中的瓶颈并不是由于运算速度不够造成的而是由于静态资源分配方法不能满足不断变化的访存需求所导致的。

带宽扩展下的性能变化比较：如图 4-6（a），在本文测试的带宽配置范围内，动态分配均表现出低于静态基线的执行延迟。在小带宽（32 bit/cycle）情况下，静态分配延时大幅上升，而动态分配还能保持较好表现，这说明在资源受限的情况下，动态资源分配可以更好地发挥出较少带宽的作用并减少由于局部资源竞争造成的访存阻塞现象，当带宽增大后，动态分配的延迟下降趋势逐渐趋缓，而静态分配仍有较大浪费等待时间开销，说明在带宽增加后，系统瓶颈逐步从单纯带宽限制转向 Bank 资源调度与访问冲突。所以单纯增加总带宽无法消除访存问题，如果资源管理方式不具备灵活性，则仍会被 Bank 利用率低下以及局部热点影响。

端口扩展下的资源利用率比较：由图 4-6（b）可以看出，随着系统端口数量从 1 增加到 4，两种方法都得到一定改善，在本文测试的端口数量范围内，动态分配均获得了不低于静态基线的性能表现，并在多数配置下具有更低访存延迟。在 4-port 的情况下，动态分配几乎消除 Stall Cycles，而静态分配仍然存在一定程度访存延迟开销，这是因为动态方法可以更好地发挥出新增加并行性所带来的好处，并且随着并行性的提高，还可以释放更多

的访存带宽；而静态分配因为其所分配资源是固定的，所以即使增加了更多的端口也无法很好地解决一些 Bank 被频繁访问的问题，所以它带来的性能提升也是有上限的。

资源均衡性和系统的可扩展性：如表 4.5 所示，由 Bank Imbalance 可以发现，在不同的设置中，动态分配都可以保证比较好的资源利用率，但是静态分配一直存在严重的资源不平衡问题。由于资源使用不合理而导致一些 Bank 长期处于较高负载情况下，进而不断发生局部冲突以及流水线堵塞的情况，这也是导致静态分配效率较低的一个重要原因。因此，动态地分配资源不仅可以减少每次内存访问时对 Bank 竞争的影响，而且还可以在整個系统范围内提高整体性能并且对于各种带宽、端口数量都是有利的。这说明我们的方法适用于复杂的 MoE 推理任务并且具有良好的可扩展性。

表 4.5 端到端实验配置与性能统计结果

Group/Strategy	带宽或端口配置	Total Cycle	Stall Cycle	Bank Imbalance
Baseline/Dynamic	-	781811	461787	6.607031
Baseline/Static	-	1870007	1549983	10.603271
BW/Dynamic	128	781811	461787	6.607031
BW/Dynamic	32	1622957	1302933	7.146429
BW/Dynamic	64	1069370	749346	7.338614
BW/Static	128	1870007	1549983	10.603271
BW/Static	32	6497318	6177294	10.603271
BW/Static	64	3385928	3065904	10.603271
Port/Dynamic	1 port	781811	461787	6.607031
Port/Dynamic	2 port	420032	100008	8.019073
Port/Dynamic	4 port	320024	0	6.392193
Port/Static	1 port	1870007	1549983	10.603271
Port/Static	2 port	836741	516717	10.603271
Port/Static	4 port	322469	2445	10.603271

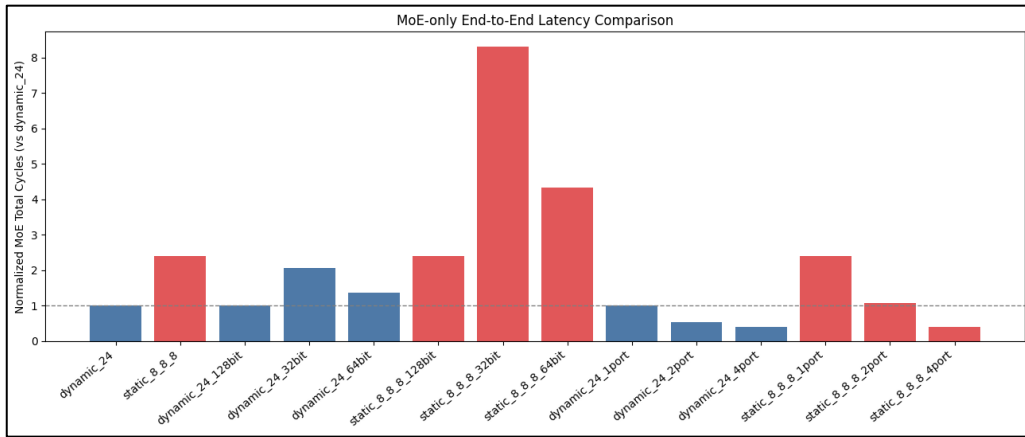
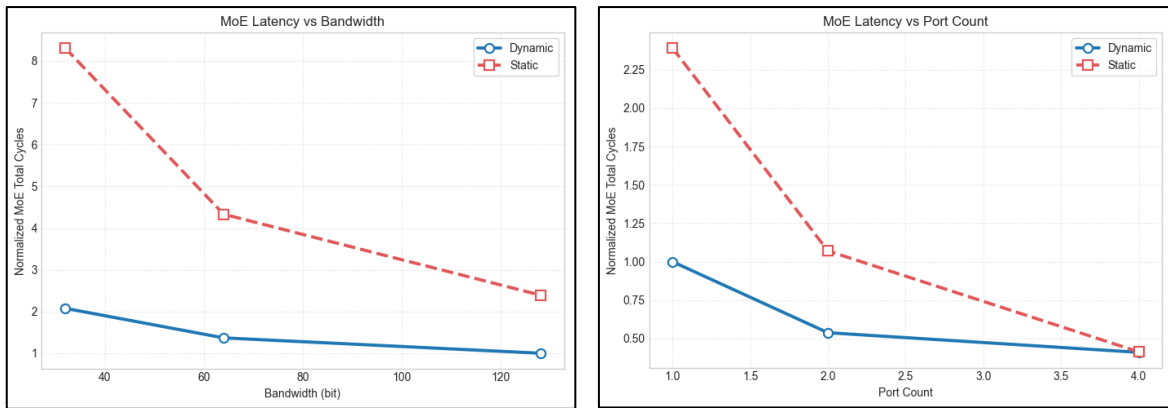


图 4-5 不同配置下端到端延迟对比



(a) 不同带宽条件下分配策略性能对比

(b) 不同 port 条件下分配策略性能对比

图 4-6 不同资源配置下分配策略的端到端性能对比

4.5 MemDomain 模块综合与实现开销分析

为了验证提出的 MemDomain 架构在硬件上是否可行,本文在 Synopsys Design Compiler 中对相关 RTL 模块进行逻辑综合以及层次化功耗分析。综合使用 28 nm 工艺库 scc28nhkcp_hdc35p140_rvt, Lib Corner 为 ssg_v0p9_m40c,而本文当前实验仅进行 DC 逻辑综合与层级功耗分析,未进行后端布局布线,因此本阶段未单独设置 RC Corner。需要说明的是,本节 DC 综合结果主要统计 MemDomain 中统一访存通路、动态 Bank 映射、请求调度、AccPipe 以及相关控制逻辑的标准单元开销,未显式接入由 SRAM Compiler 生成的 SRAM 宏单元。因此,表中面积和功耗结果不包含完整片上 SRAM 存储阵列的宏单元面积与功耗,SRAM 存储体面积在后文进行单独估算。

静态基线架构使用固定的 Bank 绑定方式,无统一访存总线,也无动态 Bank 分配以及动态映射表。在该架构中,不同访问路径按照固定规则连接到对应的 Bank 接口,普通访存请求和 Accumulator 累加请求分别经过不同的数据通路进行处理。相比之下,本文所设

计的 MemDomain 在此基础上增加了统一访存总线、动态 Bank 映射、动态调度队列以及配置管理逻辑，使上层计算单元无需关心具体物理 Bank 编号，只需通过统一接口完成片上存储访问。

MemDomain 整体综合结果见表 4.6，由表可知，在不包含 SRAM 宏单元的情况下，MemDomain 单元总面积为 $106016.400 \mu\text{m}^2$ ，其中组合逻辑所占面积为 $94518.522 \mu\text{m}^2$ ，时序逻辑所占面积为 $11497.878 \mu\text{m}^2$ ，从面积分布可以看出组合逻辑占大部分，因此该设计主要硬件开销为访存请求译码、通路选择、仲裁控制与动态调度所需要的电路，而时序逻辑主要为配置寄存器、调度队列、状态维持及流水线寄存器等；功耗上 MemDomain 总功耗为 69.548 mW ，其中动态功耗为 69.539 mW ，泄漏功耗为 $7.471 \mu\text{W}$ ，由于动态功耗较大，所以该模块主要功耗也是在运行时控制信号变化以及访存路径变化所带来的影响。

更细粒度层次划分见表 4.7。需要说明的是，表 4.7 统计的是各层级标准单元逻辑开销，而非 SRAM 宏单元面积。Backend 是 MemDomain 逻辑开销最大的部分，面积为 $61938.954 \mu\text{m}^2$ ，占总逻辑面积的 58.42% ，功耗为 60.093 mW ，占总逻辑功耗的 86.40% 。其主要开销来自 AccPipe、Bank 访问控制、映射表查询、请求路由和响应返回等逻辑，而不是 SRAM 存储阵列本身。Midend 的面积为 $23305.968 \mu\text{m}^2$ ，主要来自统一访存总线、请求汇聚和动态调度逻辑；Frontend 的面积为 $20747.286 \mu\text{m}^2$ ，主要来自访存指令解析、配置管理、读写控制和性能监视逻辑。可见，MemDomain 中的大部分开销都集中在后端和中端上，这表明动态资源分配所付出的代价主要是后端资源访问管理和统一化的访存路径。

此外，为了区分控制逻辑与存储单元对面积和功耗的影响，本文将 MemDomain 综合结果拆分为标准单元逻辑和存储相关单元两部分统计。存储相关单元主要包括 SRAM Bank 以及调度队列内部的小容量存储阵列，统计结果如表 4.8 所示。由表可知，存储相关单元总面积为 $431073.167 \mu\text{m}^2$ ，总功耗为 14.539 mW 。MemDomain 含存储相关单元的总面积为 $537089.567 \mu\text{m}^2$ ，总功耗为 84.087 mW ；扣除存储相关单元后，标准单元逻辑面积为 $106016.400 \mu\text{m}^2$ ，标准单元逻辑功耗为 69.548 mW 。因此，表 4.6、表 4.7 和后续的表 4.9 均采用标准单元逻辑统计口径，以反映统一访存通路、动态 Bank 映射和动态调度机制带来的控制逻辑开销。

新旧架构对比见表 4.9。静态基线架构标准单元逻辑面积为 $91348.702 \mu\text{m}^2$ ，本文研究架构标准单元逻辑面积为 $106016.400 \mu\text{m}^2$ ，增加了约 16.06% ，其中组合逻辑面积从 $81348.951 \mu\text{m}^2$ 增加到 $94518.522 \mu\text{m}^2$ ，增加了约 16.19% ，时序逻辑面积从 $9999.750 \mu\text{m}^2$ 增加到 $11497.878 \mu\text{m}^2$ ，增加了约 14.98% ，由于有动态 Bank 映射、资源状态维护以及调度队列等原因造成了更多的寄存器。

从功耗来看，静态基线架构标准单元逻辑功耗是 61.621 mW，而本文研究架构标准单元逻辑功耗是 69.548 mW，上升了约 12.86%，其中动态功耗从 61.614 mW 上升到 69.539 mW，上升约 12.86%，泄漏功耗从 6.568 μ W 上升到 7.471 μ W，上升约 13.75%。这说明新的设计方案带来的额外功耗大部分来自于统一总线仲裁、动态映射、请求排队及多路径的选择这些控制单元工作时产生的翻转。此外虽然泄漏功耗也有一定幅度增长，但是由于数值很小所以对于整体功耗的影响不大。

进一步参考前面 MoDSE 工作负载级实验可以看出，在一定标准单元面积以及逻辑功耗代价情况下，本文所提方案仍然具有较大优势。相同工作负载与硬件条件下，静态 Baseline 架构总的时钟周期数为 1870007，而使用动态预取协同优化后总的时钟周期数为 541827，减少大约 71.03%，相当于加速比达到 3.45 $\times$ ；访存等待时钟周期从 1549983 减少到 221803，减少了约 85.69%，即改善比达到 6.99 $\times$ 。由此可见，本文所做的工作是在付出一定标准单元逻辑面积以及逻辑功耗代价基础上获得较大收益，即良好性能一代价权衡。

从以上结果可见，在不计入 SRAM 宏单元的标准单元逻辑统计口径下，本文架构相较静态基线架构带来一定面积和功耗开销，主要来自统一访存总线、动态 Bank 映射、动态调度队列以及后端接口适配等逻辑。由于两种方案采用相同的 SRAM 容量配置，因此逻辑开销对比能够反映本文动态资源分配机制相对于静态基线所引入的额外控制成本。结合前文性能实验可以看出，在本文实验配置和所选静态基线范围内，该额外逻辑开销能够换取更低的 Bank 冲突和更高的片上存储资源利用率。

表 4.6 MemDomain 标准单元逻辑综合结果统计

指标	数值
工艺	28 nm
Lib_Corner	ssg_v0p9_m40c
时钟周期	2 ns
Total Cell Area	106016.400 μm^2
Combinational Area	94518.522 μm^2
Sequential Area	11497.878 μm^2
Total Power	69.548 mW
Dynamic Power	69.539 mW
Leakage Power	7.471 μ W

表 4.7 MemDomain 层级标准单元面积与功耗统计

模块	面积/ μm^2	面积占比	功耗/mW	功耗占比
Backend	61938.954	58.42%	60.093	86.40%
Midend	23305.968	21.99%	3.052	4.39%
Frontend	20747.286	19.57%	6.403	9.21%
Total	106016.400	100.00%	69.548	100.00%

表 4.8 DC 层级报告中存储宏单元统计结果

项目	数量	面积/ μm^2	功耗/mW
mem_128x128	32	292539.062	4.898
mem_64x128	16	124363.328	2.429
ram_4x161	1	12269.872	6.298
ram_2x4	1	1900.904	0.920
Total	—	431073.167	14.539

表 4.9 新旧架构综合结果对比

指标	静态基线架构	动态预取架构（本文）	变化比例
Total Cell Area/ μm^2	91348.702	106016.400	+16.06%
Combinational Area/ μm^2	81348.951	94518.522	+16.19%
Sequential Area/ μm^2	9999.750	11497.878	+14.98%
Dynamic Power/mW	61.614	69.539	+12.86%
Leakage Power/ μW	6.568	7.471	+13.75%
Total Power/mW	61.621	69.548	+12.86%
Total Cycle	1870007	541827	-71.03%
Stall Cycle	1549983	221803	-85.69%

第五章 总结与展望

5.1 工作总结

本文主要针对 MoE 模型在推理时所遇到 Memory-Bound 问题,在硬件层面对其片上存储系统进行了一系列的研究以及优化工作。

首先,从对 MoE 模型访存行为建模及分析可以看出,它相比于传统的稠密模型,在存储访问方面有较大区别。实验表明,MoE 层占用了整个系统的大部分开销,它的 Stall Cycles 以及 Bank 冲突数都远大于非 MoE 层。而且更进一步说明,由于传统的静态 Bank 分配方式不能有效应对 MoE 模型不断变化的访存要求,所以造成低效的原因主要是此问题。

为了解决以上问题,我们提出一种基于总线统一以及存储资源虚拟化片上内存结构 (MemDomain),在这种结构中统一 Accumulator 和 SRAM 访问路径,避免因多路径存在而导致额外开销;另外增加虚拟 Bank 管理方式使得实际物理存储空间可以被灵活地分配给不同 Bank 使用从而更好地支持非规则访存模式。此外还提供配置相关以及数据传输自定义指令以实现软硬件联合优化。

在实现与验证方面,本文在 Buckyball 平台上完成了 MemDomain 架构的 RTL 级设计,并通过 BBtest 测试工具对 mset、mvin、mvout 等自定义指令以及虚拟 Bank 到物理 Bank 的动态映射过程进行了功能验证,证明了所设计结构在数据搬运、资源配置、多 Bank 写回和结果校验等关键路径上的正确性。在性能评估方面,本文基于周期级仿真对动态资源分配机制进行测试。实验结果表明,相比静态分配方案,动态 Bank 分配可使 MoE 层推理延迟降低 58.19%,访存延迟降低 70.21%;在进一步引入预取窗口后,动态分配与预取协同方案相比静态无预取基线可使端到端执行周期降低 71.03%,有效访存延迟降低 85.69%。在硬件开销方面,本文使用 Synopsys Design Compiler 对 MemDomain 逻辑部分进行综合。在 28 nm 工艺、2 ns 时钟周期条件下,相比静态基线架构,本文架构标准单元逻辑面积增加 16.06%,逻辑功耗增加 12.86%。综合来看,本文提出的动态资源分配与预取协同机制能够在较小的逻辑开销下有效缓解 Bank 冲突,降低访存延迟,并提升片上存储资源利用率。

总之,本文提出的片上内存总线统一以及虚拟化的设计方案可以有效缓解 MoE 模型所面临 Memory-Bound 的问题。在此基础上,本文的创新点可总结为:(1)提出了统一总

线的片上存储架构，实现多路径访存的整合及简化；（2）采用虚拟 Bank 动态分配的方法来提升存储资源的使用效率以及解决 Bank 冲突问题；（3）设计软硬件协同的存储调度策略，实现灵活高效的片上存储管理。

5.2 工作展望

尽管本文在 MoE 模型片上存储体系结构优化方面取得了一定成果，但仍存在进一步改进与拓展的空间，未来工作可从以下几个方向展开：

（1）更精细的动态调度策略

本文当前的动态资源分配主要基于运行时访存需求进行调度，但仍属于相对粗粒度的策略。未来可以结合更细粒度的访问预测机制，引入基于历史行为或机器学习的调度算法，实现更加智能的资源分配。

（2）更细粒度的预取预测机制

本文主要从片上存储资源管理角度研究预取与动态 Bank 分配的协同关系，通过预取窗口和虚拟 Bank 映射机制减少数据搬运带来的访存等待。后续工作可以进一步结合专家激活概率、Token 路由历史和层间访存相关性，设计更细粒度的专家级或 Token 级预取预测机制，从而进一步提高预取准确性并降低无效预取带来的资源占用。

（3）多核与跨核共享存储扩展

当前设计主要面向单核或单加速器场景，未来可以扩展至多核环境，研究共享存储资源的动态分配与一致性问题，以支持更大规模的分布式 MoE 推理。

（4）更真实的系统级建模

本文在仿真中主要关注片上存储系统，未考虑完整 DRAM 时序及缓存层级。未来可以结合更完整的内存层级模型，对系统性能进行更全面评估。

（5）面向实际应用的部署验证

后续可将所提出架构应用于真实 AI 推理任务中，结合实际工作负载验证其性能提升效果，从而增强方案的工程应用价值。

参考文献

- [1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. 2017: 5998-6008.
- [2] DEVLIN J, CHANG M W, LEE K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding[R/OL]. arXiv:1810.04805, 2018[2026-05-27]. <https://arxiv.org/abs/1810.04805>.
- [3] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[R/OL]. arXiv:2005.14165, 2020[2026-05-27]. <https://arxiv.org/abs/2005.14165>.
- [4] JOUPPI N P, YOUNG C, PATIL N, et al. In-datacenter performance analysis of a tensor processing unit[C]//Proceedings of the 44th Annual International Symposium on Computer Architecture. New York: ACM, 2017: 1-12.
- [5] SAMAJDAR A, ZHU Y H, WHATMOUGH P, et al. SCALE-Sim: Systolic CNN accelerator simulator[R/OL]. arXiv:1811.02883, 2018[2026-05-27]. <https://arxiv.org/abs/1811.02883>.
- [6] RAJ R, BANERJEE S, CHANDRA N, et al. SCALE-Sim v3: A modular cycle-accurate systolic accelerator simulator for end-to-end system analysis[R/OL]. arXiv:2504.15377, 2025[2026-05-27]. <https://arxiv.org/abs/2504.15377>.
- [7] SHAZEER N, MIRHOSEINI A, MAZIAR R, et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer[R/OL]. arXiv:1701.06538, 2017[2026-05-27]. <https://arxiv.org/abs/1701.06538>.
- [8] ZOPH B, BELLO I, KUMAR S, et al. ST-MoE: Designing stable and transferable sparse expert models[R/OL]. arXiv:2202.08906, 2022[2026-05-25]. <https://arxiv.org/abs/2202.08906>.
- [9] YU Z K, GUAN Y, YU Z H, et al. Patterns behind Chaos: Forecasting Data Movement for Efficient Large-Scale MoE LLM Inference [R/OL]. arXiv:2510.05497, 2025[2026-05-27]. <https://arxiv.org/abs/2510.05497>.
- [10] LEPIKHIN D, LEE H, XU Y, et al. GShard: Scaling giant models with conditional computation and automatic sharding[C]//International Conference on Learning Representations. 2021.
- [11] FEDUS W, ZOPH B, SHAZEER N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity[J]. Journal of Machine Learning Research, 2022, 23(120): 1-39.
- [12] DU N, HUANG Y, DAI A M, et al. GLaM: Efficient scaling of language models with mixture-of-experts[C]//Proceedings of the 39th International Conference on Machine Learning. PMLR, 2022, 162: 5547-5569.

-
- [13] WILLIAMS S, WATERMAN A, PATTERSON D. Roofline: An insightful visual performance model for multicore architectures[J]. Communications of the ACM, 2009, 52(4): 65-76.
 - [14] ASAD A, KAUR R, MOHAMMADI F. A survey on memory subsystems for deep neural network accelerators[J]. Future Internet, 2022, 14(5): 146.
 - [15] KIM D, KUNG J, CHAI S, et al. Neurocube: A programmable digital neuromorphic architecture with high-density 3D memory[J]. ACM SIGARCH Computer Architecture News, 2016, 44(3): 380-392.
 - [16] KIM D, NA T, YALAMANCHILI S, et al. DeepTrain: A programmable embedded platform for training deep neural networks[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 37(11): 2360-2370.
 - [17] UEYOSHI K, ANDO K, HIROSE K, et al. QUEST: A 7.49TOPS multi-purpose log-quantized DNN inference engine stacked on 96MB 3D SRAM using inductive-coupling technology in 40nm CMOS[C]//Proceedings of the IEEE International Solid-State Circuits Conference. Piscataway: IEEE, 2018: 216-218.
 - [18] IMANI M, GUPTA S, KIM Y, et al. FloatPIM: In-memory acceleration of deep neural network training with high precision[C]//Proceedings of the 46th International Symposium on Computer Architecture. 2019: 802-815.
 - [19] ANGIZI S, HE Z, FAN D. ParaPIM: A parallel processing-in-memory accelerator for binary weight deep neural networks[C]//Proceedings of the Asia and South Pacific Design Automation Conference. 2019: 127-132.
 - [20] LI J J, YAN G H, LU W Y, et al. SmartShuttle: Optimizing off-chip memory accesses for deep learning accelerators[C]//Proceedings of the Design, Automation and Test in Europe Conference. 2018: 343-348.
 - [21] PUTRA R V, HANIF M A, SHAFIQUE M. ROMANet: Fine-grained reuse-driven off-chip memory access management and data organization for deep neural network accelerators[J]. IEEE Transactions on Very Large Scale Integration Systems, 2021, 29: 702-715.
 - [22] YU Y P, MA H Y, AGARWAL K, et al. Efficient MoE serving in the memory-bound regime: Balance activated experts, not tokens[R/OL]. arXiv:2512.09277, 2025[2026-05-27]. <https://arxiv.org/abs/2512.09277>.
 - [23] HWANG R, WEI J, CAO S, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference[C]//Proceedings of the 51st Annual International Symposium on Computer Architecture. Piscataway: IEEE, 2024: 1018-1031.
 - [24] GENC H, KIM S, AMID A, et al. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration[C]//Proceedings of the 58th ACM/IEEE Design Automation Conference. Piscataway: IEEE, 2021: 769-774.

- [25] SUN M X, LIU W, LUAN J, et al. Mixture of Diverse Size Experts[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track. Miami: Association for Computational Linguistics, 2024: 1608-1621.
- [26] DANGOSYS. Buckyball: An open-source scaling framework for domain-specific architectures[CP/OL]. [2026-05-27]. <https://github.com/DangoSys/buckyball>.

致 谢

在此次毕业设计中，我得到了两位老师以及组里师兄的帮助与指导，在这里向大家致以诚挚谢意。

首先，我要向我的导师单伟伟老师和江哲老师表示最深切的感谢，在论文选题、研究方法以及论文写作中都获得他们的悉心关怀和指导，从最初的课题选定，到后来的整体框架搭建以及具体实施细节把握，在此期间老师们一直严格要求我不断进步提高自己，让我获益良多。

其次，感谢实验室王晖师兄在项目进行期间提供的帮助。在 MemDomain 架构理解上，在 Buckyball 平台上操作及实验环境配置上，王晖师兄给予许多指导性意见使得我能较好地完成相关工作，在不定期向其请教问题时也使我对体系结构设计有了更进一步了解。此外，感谢阎智昊师兄在论文修改过程中提出了大量细致而宝贵的意见。师兄以认真负责、严谨细致的科研态度，花费宝贵时间逐字逐句地帮助我完善论文内容，使我在论文结构、语言表达和学术规范等方面都受益良多。在此向两位师兄表示衷心的感谢。

最后，感谢东南大学电子科学与工程学院，在我本科期间奠定良好专业基础。本次毕业设计使我提高自身工程实践水平，同时对于计算机体系结构以及人工智能加速器设计有更进一步了解。

谨以此文献给所有关心、支持、帮助过我的老师、同学以及朋友们。

止於至善



SOUTHEAST UNIVERSITY